Balanced Search Trees

Chapter 19

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

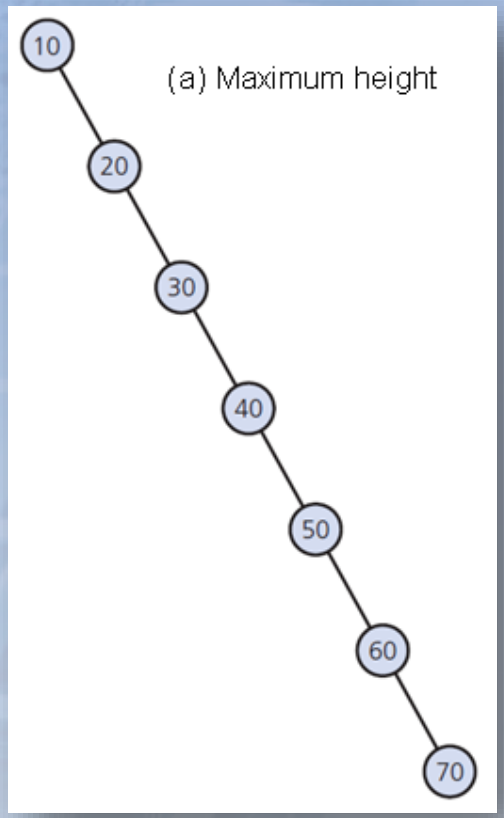
Balanced Search Trees

- Height of binary search tree
  - Sensitive to order of additions and removals
- Various search trees can retain balance
  - Despite additions and removals

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Balanced Search Trees

If you inserted the items: 10, 20, 30, 40, 50, 60, and 70 into the tree in ascending order, you would obtain a binary search tree of maximum height



If you inserted the items: 40, 20, 60, 10, 30, 50, and 70 into the tree in that order, you would obtain a binary search tree of minimum height

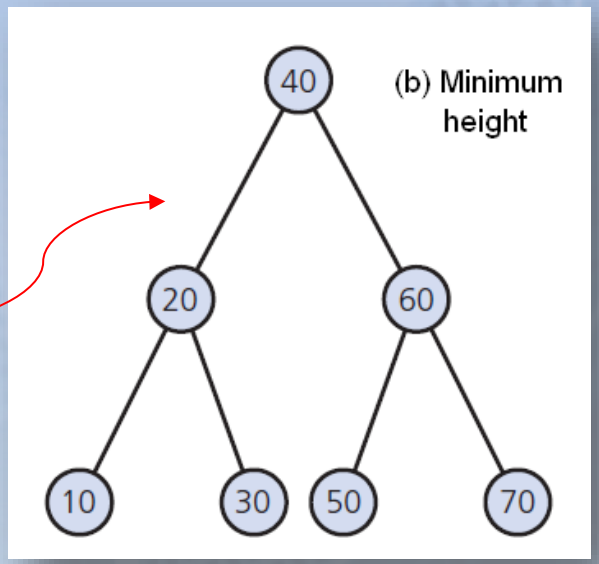


FIGURE 19-1 The tallest and shortest binary search trees containing the same data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Evolution of Self-Balancing Trees

Year	Type	Rank (Usage)	Key Feature	Highlights
1962	AVL Tree	5	• First self-balancing BST • Balances via height difference (balance factor ±1) • Rotation-heavy	Early, rigidly balanced binary search tree, often used in compilers or memory managers
1970	2-3 Tree	3	• Multi-way search tree • Nodes can hold 1 or 2 keys (2 or 3 children) • Always height-balanced	Used in theoretical data structures, academic applications, some tree-based indexers
1972	B-Trees	2	• Generalized form for databases/disks • Balanced tree for block-based storage	Standard in databases (PostgreSQL, MySQL, SQLite), file systems (NTFS, HFS+)
1973	Red-Black Tree	1	• Simulates 2-3-4 Trees using binary representation • Efficient and widely used	Widely used in C++ STL (std::map, std::set), Java (TreeMap, TreeSet), Linux kernel
1974	2-3-4 Tree	4	• Extension of 2-3 trees with 2, 3, or 4 children • Maps directly to Red-Black Trees	Extends 2-3 Tree with 4-node capability, often used in databases and file systems
1978	Splay Tree	6	• Self-adjusting, no balance info stored • Frequently accessed elements are faster	Used in specialized systems (e.g., caches), but rarely in general-purpose libraries
1989	Treap	7	• Combines BST and heap using random priorities • Probabilistic balancing	Used in randomized algorithms or competitive programming; rare in real-world applications
1999	AA Tree	9	• Simplified Red-Black Tree • Easier to implement but slightly less efficient	Simplified variant of Red-Black Trees, used in educational contexts or hobby projects
2002	Scapegoat Trees	8	• Alternative balancing schemes • Good worst-case guarantees in certain scenarios	Theoretically elegant but rarely used in production due to complex rebalancing
2002	Weight-balanced	10	• Alternative balancing schemes • Good worst-case guarantees in certain scenarios	Mostly academic or in specific algorithm design contexts, rarely used in industry

Chatgpt.com - can you please provide a chronology/evolution of the most common self-balancing trees, including 2-3 and 2-3-4 trees?

Binary Search Tree

https://en.wikipedia.org/wiki/Binary_search_tree

Binary search tree		
Type	tree	
Invented	1960	
Invented by	P.F. Windley, A.D. Booth, A.J.T. Colin, and T.N. Hibbard	
Time complexity in big O notation		
Operation	Average	Worst case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Space complexity		
Space	$O(n)$	$O(n)$

- The AVL tree was invented by Georgy Adelson-Velsky and Evgenii Landis in 1962 for the efficient organization of information. It was the first self-balancing binary search tree to be invented.
- Various height-balanced binary search trees were introduced to confine the tree height, such as AVL trees (1962), Treaps (1989) (https://en.wikipedia.org/wiki/Treap), and red–black trees (1978) (https://en.wikipedia.org/wiki/Red%E2%80%93black_tree).

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

- An AVL tree
  - A balanced binary search tree
- Maintains its height close to the minimum
- Rotations restore the balance

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees – Balance Factor

- In a binary tree the **balance factor** of a node X is defined to be the height difference of its two child sub-trees rooted by node X.
- The subtree heights are stored at each node for all nodes in an AVL Tree, and the balance factor is calculated based on its subtree heights to check if the tree has become out of balance.
- The height of a subtree is the number of edges between the root node of the subtree and the leaf node farthest down in that subtree.
- Balance factor values
  - 0: The node is in balance.
  - more than 0: The node is "right heavy".
  - less than 0: The node is "left heavy".
- **If the balance factor is less than -1, or more than 1**, for one or more nodes in the tree, the tree is considered not in balance, and a rotation operation is needed to restore balance.

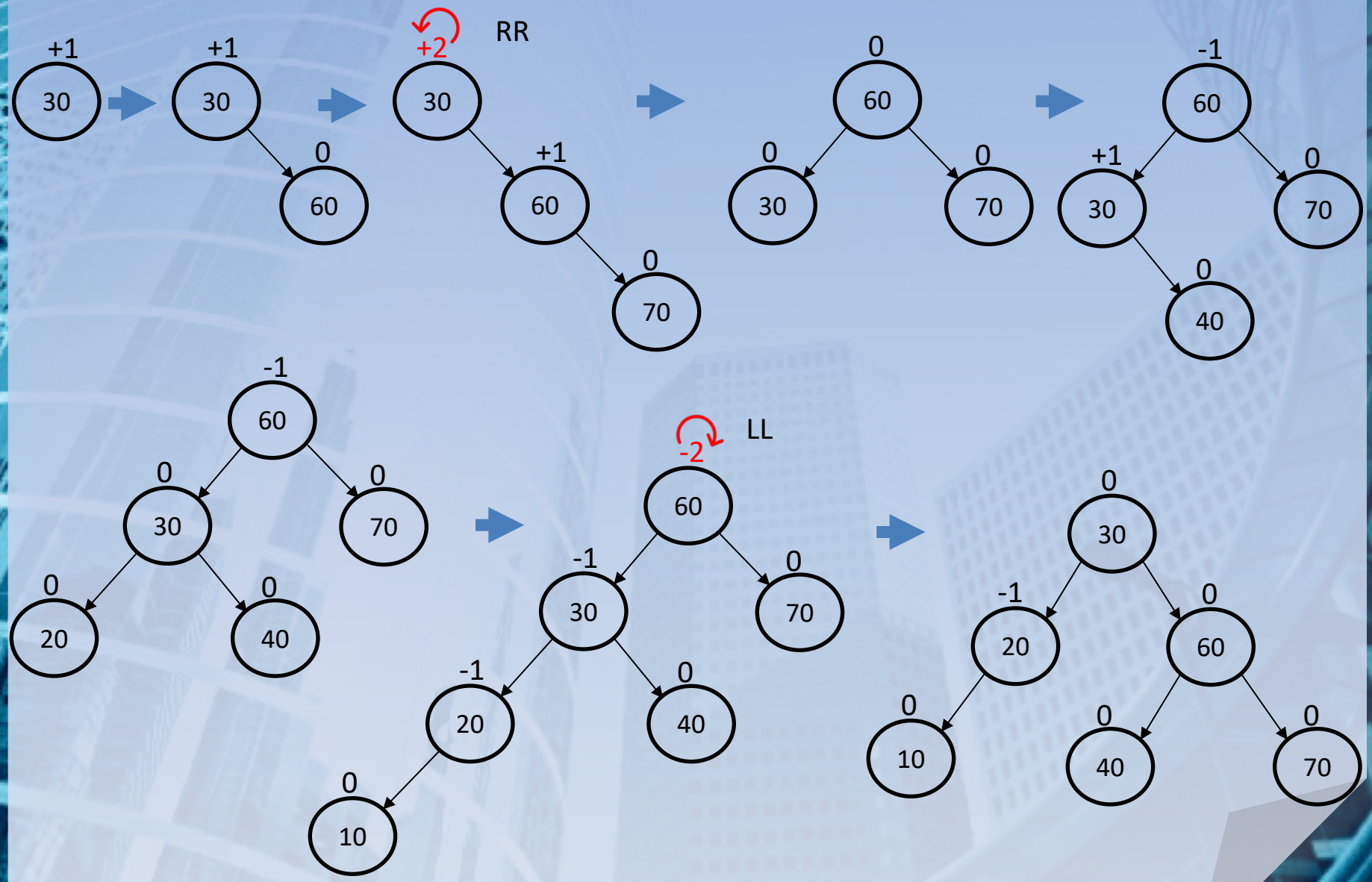
https://en.wikipedia.org/wiki/AVL_tree
https://www.w3schools.com/dsa/dsa_data_avltrees.php

AVL Trees – Out-of-Balance Cases

Case	Description	Rotation to Restore Balance
Left-Left (LL)	The unbalanced node and its left child node are both left-heavy.	A single right rotation.
Right-Right (RR)	The unbalanced node and its right child node are both right-heavy.	A single left rotation.
Left-Right (LR)	The unbalanced node is left heavy, and its left child node is right heavy.	First do a left rotation on the left child node, then do a right rotation on the unbalanced node.
Right-Left (RL)	The unbalanced node is right heavy, and its right child node is left heavy.	First do a right rotation on the right child node, then do a left rotation on the unbalanced node.

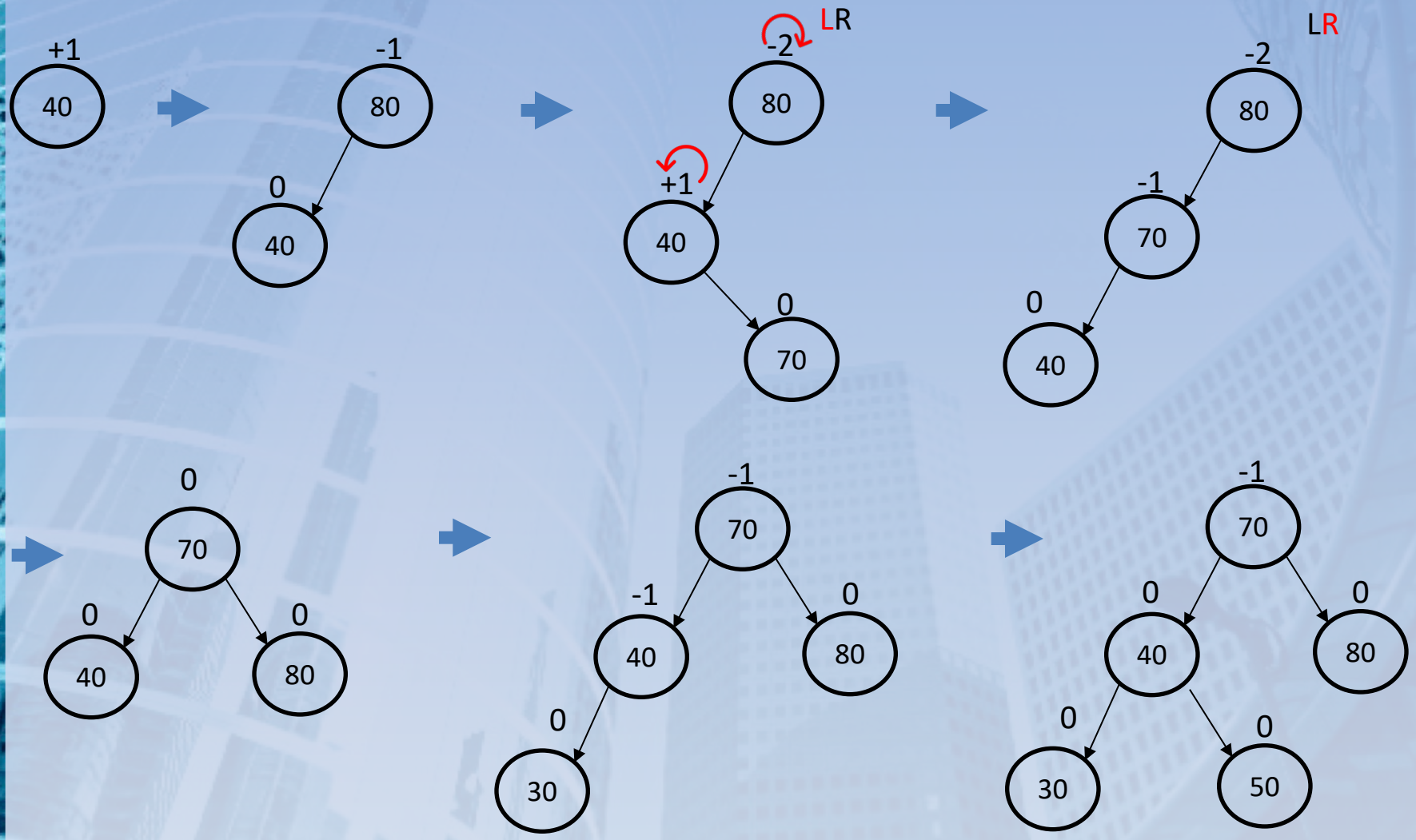
https://en.wikipedia.org/wiki/AVL_tree
https://www.w3schools.com/dsa/dsa_data_avltrees.php

AVL Trees – Left-Left (LL) and Right-Right (RR)



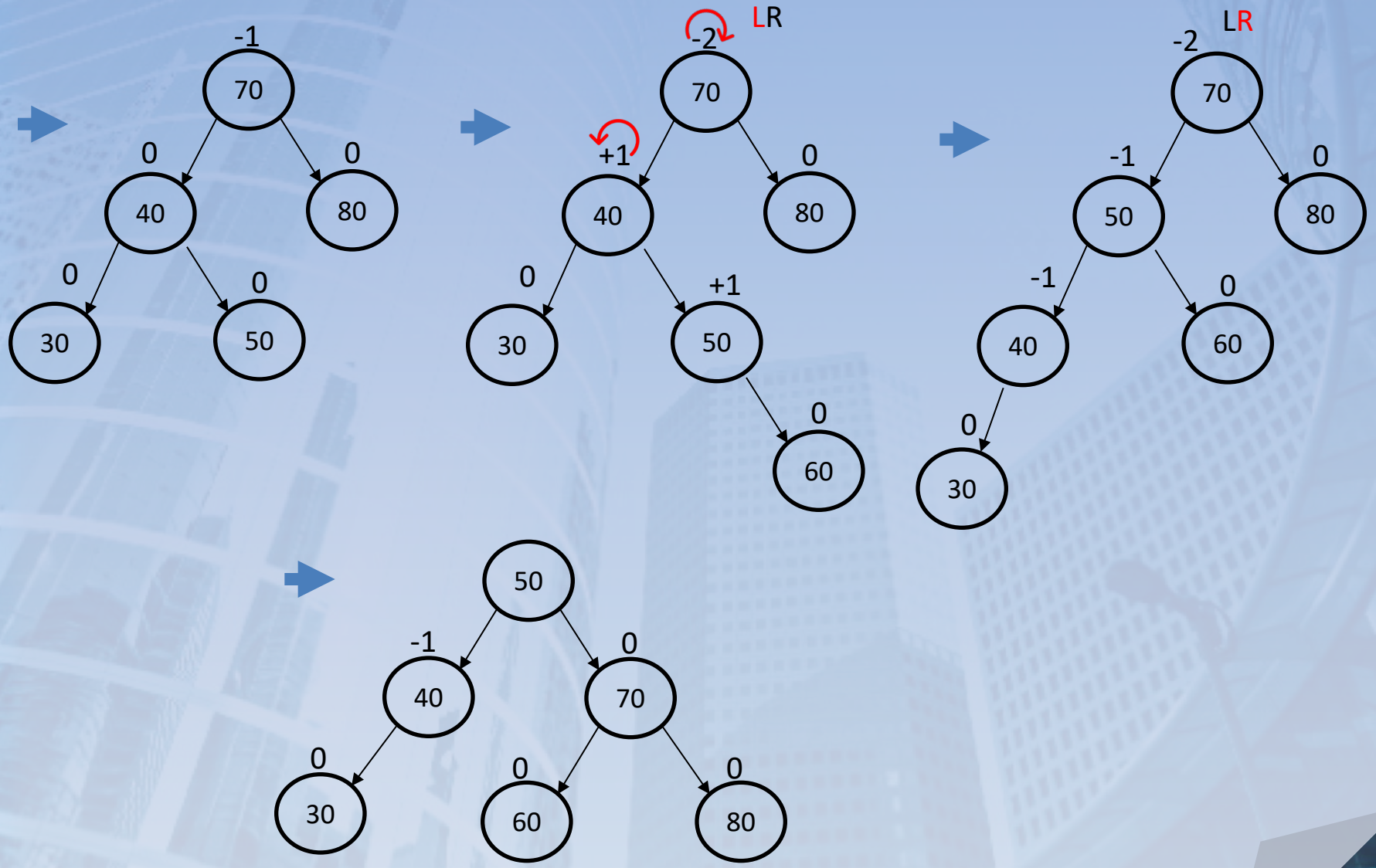
AVL – add 30, 60, 70, 40, 20, and 10

AVL Trees – Left-Right (LR)



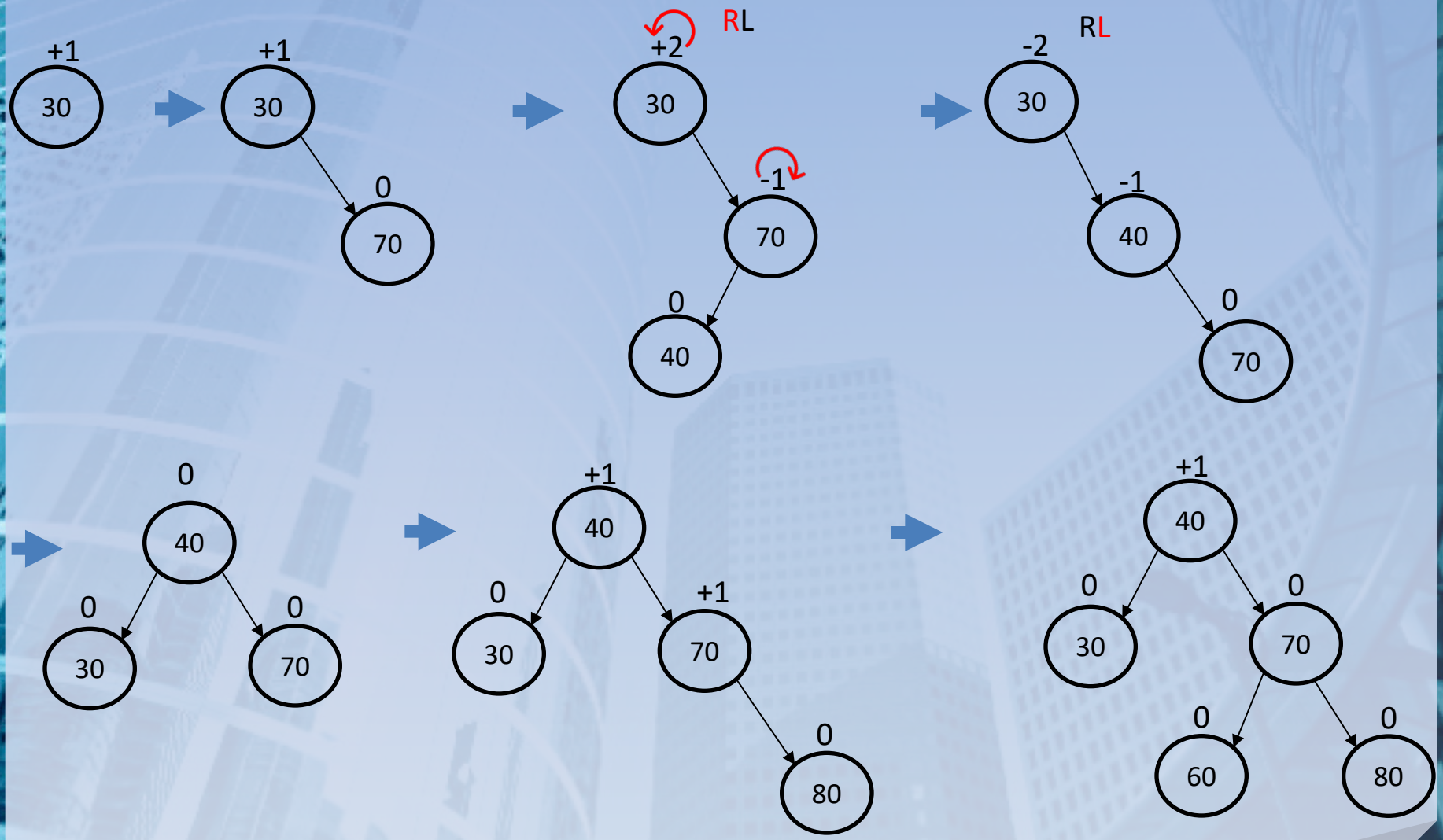
AVL – add 40, 80, 70, 30, 50, 60

AVL Trees – Left-Right (LR) (cont.)



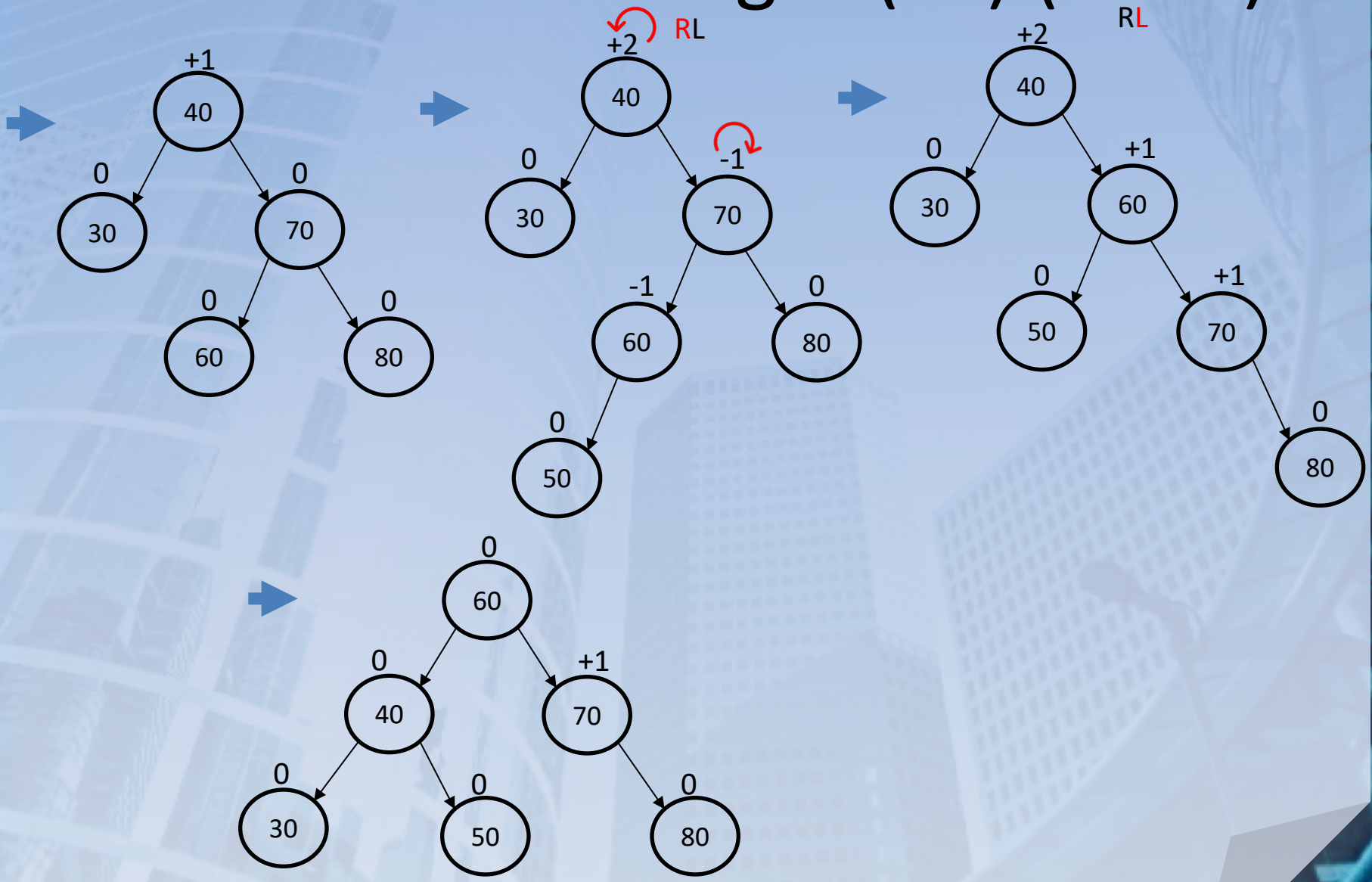
AVL – add 40, 80, 70, 30, 50, 60

AVL Trees – Left-Right (RL)



AVL – add 30, 70, 40, 80, 60, 50

AVL Trees – Left-Right (RL) (cont.)



AVL – add 30, 70, 40, 80, 60, 50

AVL Trees

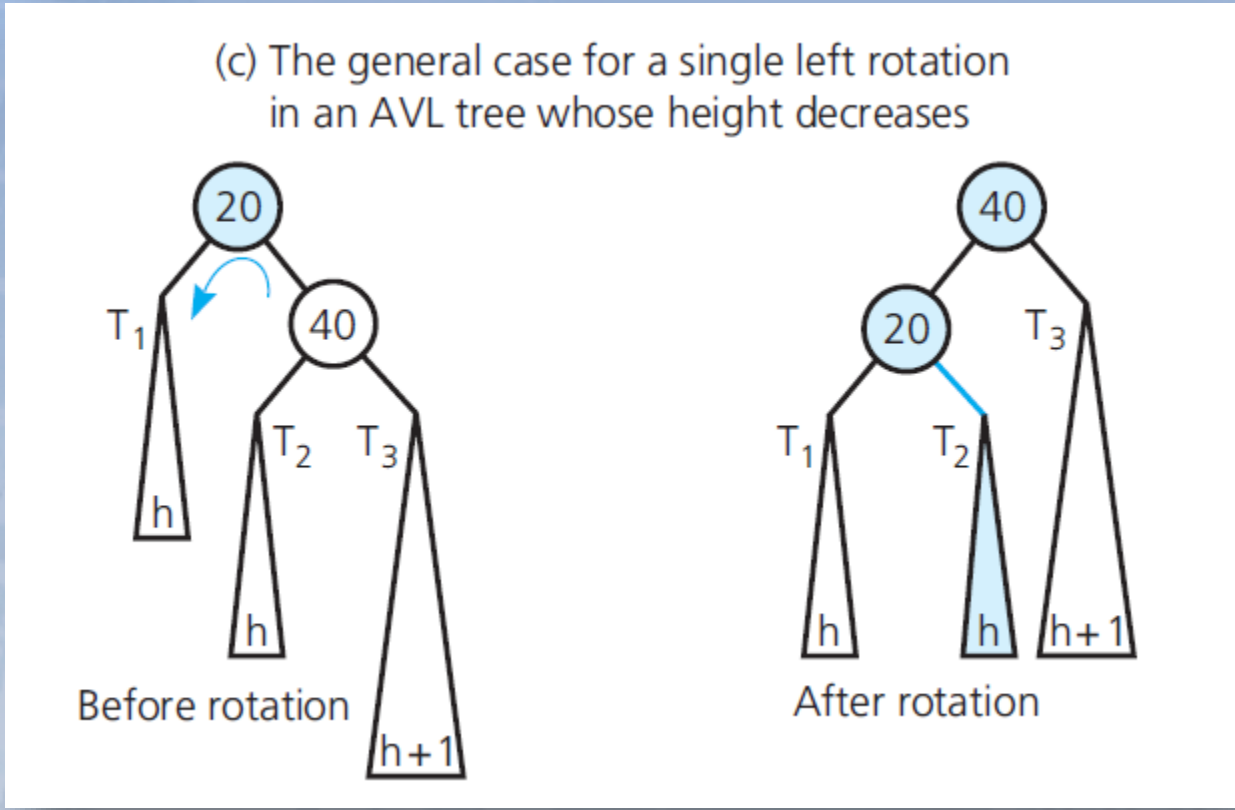


FIGURE 19-3 Correcting an imbalance in an AVL tree due to an addition by using a single rotation to the left

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

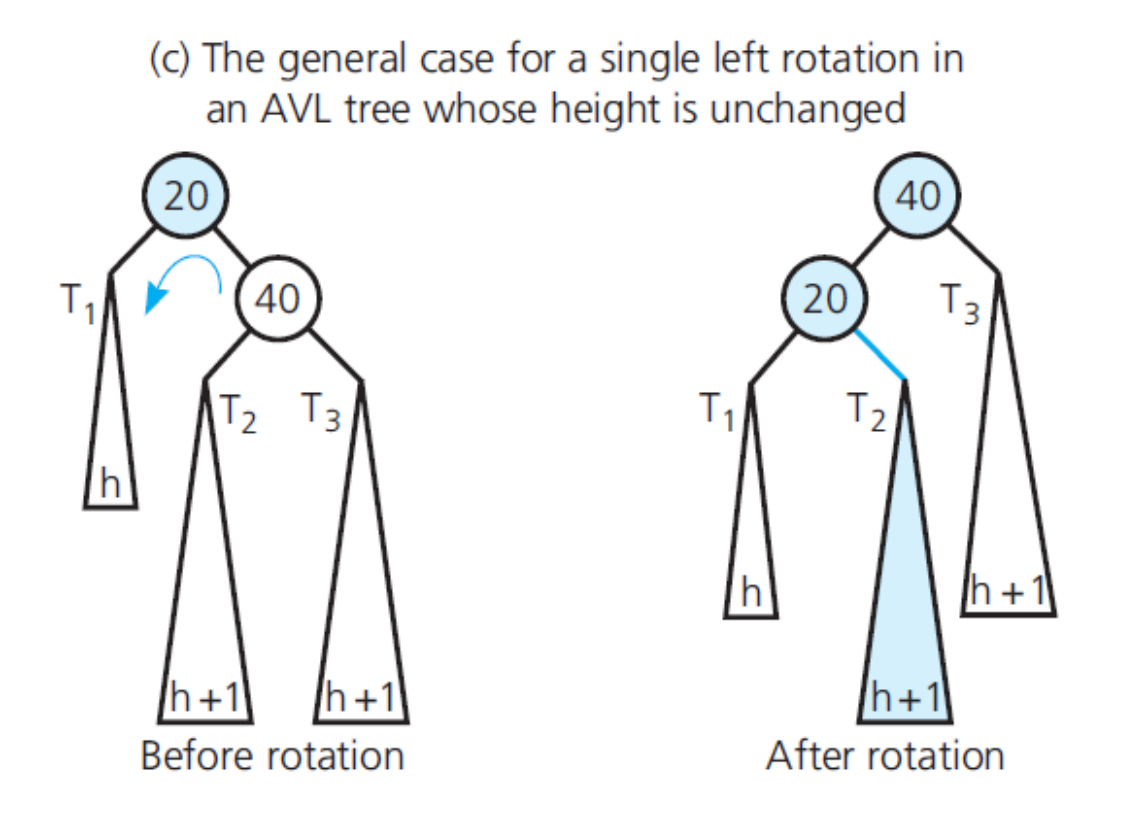


FIGURE 19-4 A single rotation to the left that does not affect the height of an AVL tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

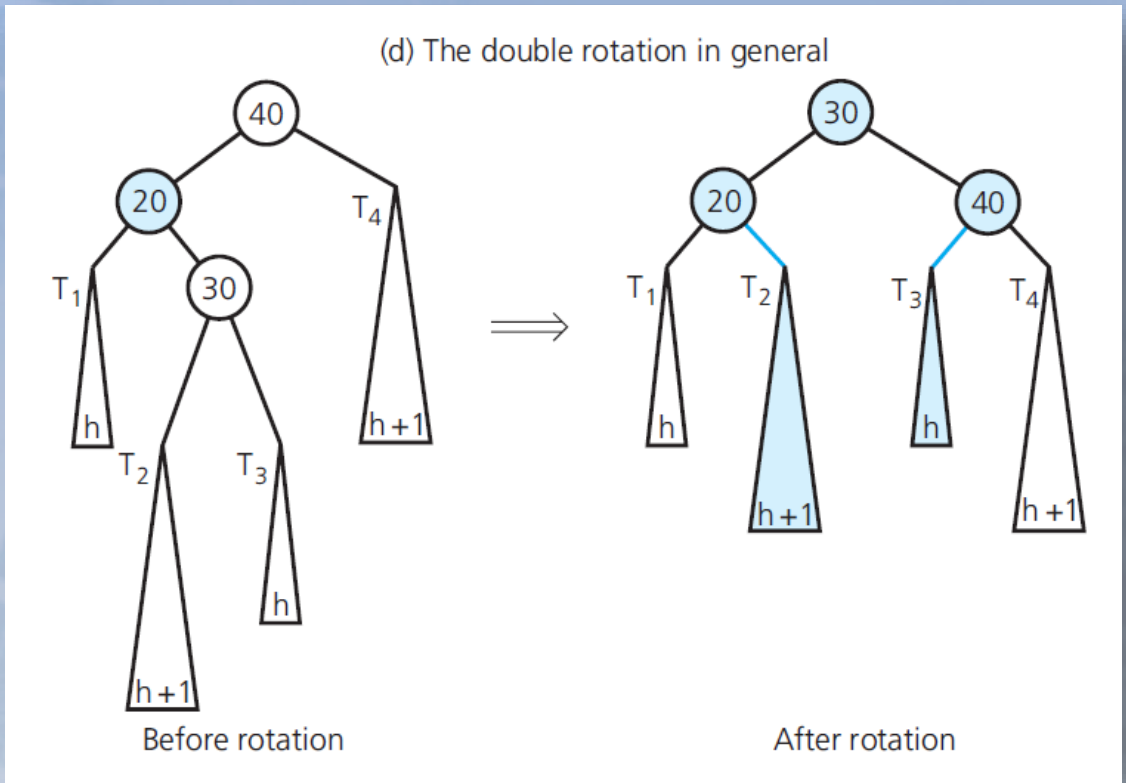


FIGURE 19-5 A double rotation that decreases the height of an AVL tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

2–3 tree		
Type	tree	
Invented	1970	
Invented by	John Hopcroft	
Complexities in big O notation		
	Space complexity	
Space	$O(n)$	
	Time complexity	
Function	Amortized	Worst Case
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

https://en.wikipedia.org/wiki/2%E2%80%933_tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

- A 2-3 tree is a tree in which each internal node (nonleaf) has either two or three children
- All leaves are at the same level; the tree is height balanced
- Data is kept in sorted order, with left subtree containing smaller values and right subtree containing larger values
- A node with two children is called a 2-node
- A node with three children is called a 3-node

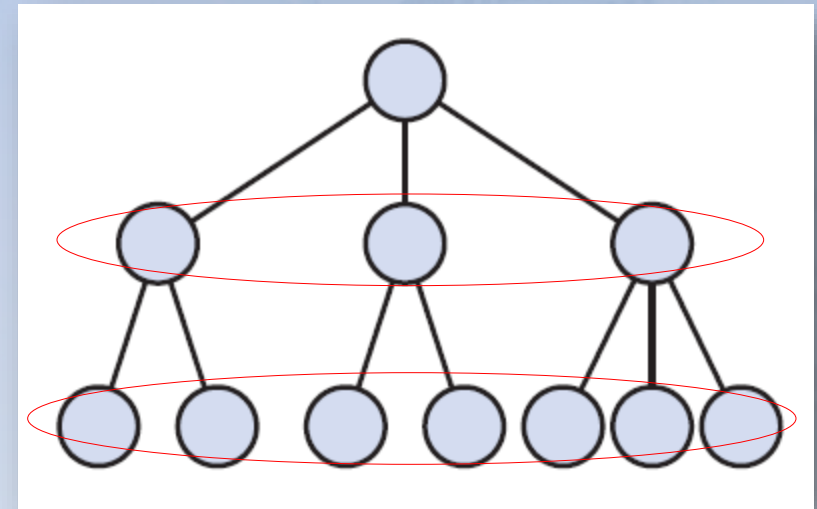


FIGURE 19-6 A 2-3 tree of height 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

- A 2-node, which has two (2) children, must contain a single data item that is > than the left child's item and < the right child's item
- A 3-node, which has three (3) children must contain two (2) data items, S and L, that satisfy the following relationship: S is > the left child's item(s) and < than the middle child's item(s); L > than the middle child's item(s) and < the right child's item(s)
- A leaf may contain either one (1) or two (2) data items.

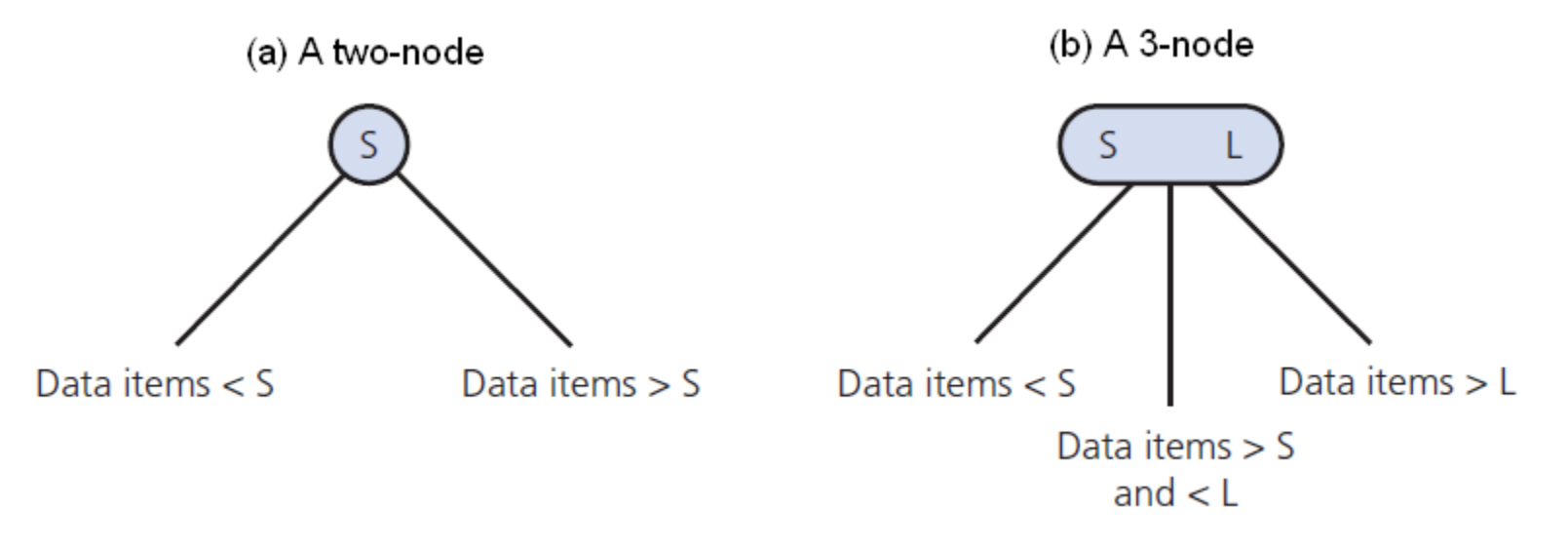


FIGURE 19-7 Nodes in a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

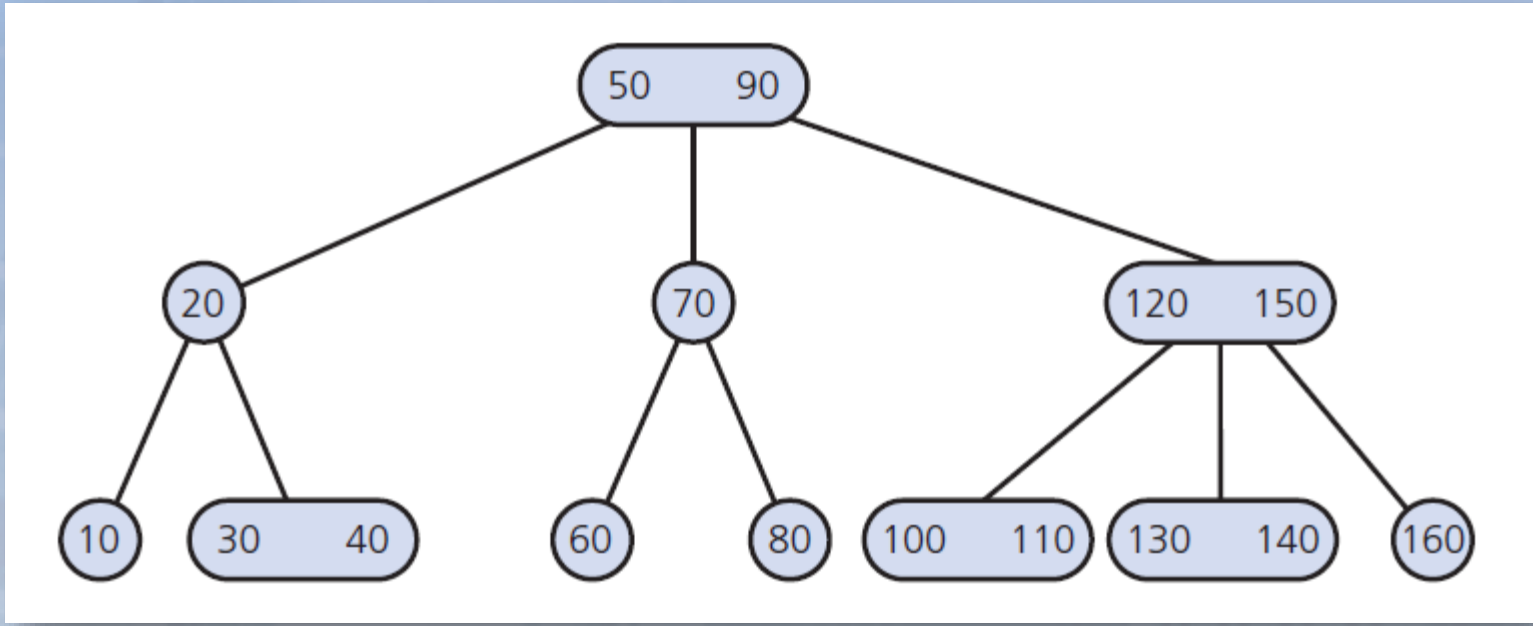


FIGURE 19-8 A 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

```
/** A class of nodes for a link-based 2-3 tree.
 @file TriNode.h */

#ifndef TRI_NODE_
#define TRI_NODE_

template<class ItemType>
class TriNode
{
private:
   ItemType smallItem;                                  // Data portion
   ItemType largeItem;                                  // Data portion
   std::shared_ptr<TriNode<ItemType>> leftChildPtr;   // Left-child pointer
   std::shared_ptr<TriNode<ItemType>> midChildPtr;    // Middle-child pointer
   std::shared_ptr<TriNode<ItemType>> rightChildPtr;  // Right-child pointer

public:
   TriNode();
```

LISTING 19-1 A header file for a class of nodes for a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

```
    bool isLeaf() const;
    bool isTwoNode() const;
    bool isThreeNode() const;

    ItemType getSmallItem() const;
    ItemType getLargeItem() const;

    void setSmallItem(const ItemType& anItem);
    void setLargeItem(const ItemType& anItem);
    auto getLeftChildPtr() const;
    auto getMidChildPtr() const;
    auto getRightChildPtr() const;

    void setLeftChildPtr(std::shared_ptr<TriNode<ItemType>> leftPtr);
    void setMidChildPtr(std::shared_ptr<TriNode<ItemType>> midPtr);
    void setRightChildPtr(std::shared_ptr<TriNode<ItemType>> rightPtr);
}; // end TriNode
#include "TriNode.cpp"
#endif
```

LISTING 19-1 A header file for a class of nodes for a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversing a 2-3 Tree

```
// Traverses a nonempty 2-3 tree in sorted order.
inorder(23Tree: TwoThreeTree): void
{
   if (23Tree's root node r is a leaf)
      Visit the data item(s)
   else if (r has two data items)
   {
      inorder(left subtree of 23Tree's root)
      Visit the first data item
      inorder(middle subtree of 23Tree's root)
      Visit the second data item
      inorder(right subtree of 23Tree's root)
   }
   else // r has one data item
   {
      inorder(left subtree of 23Tree's root)
      Visit the data item
      inorder(right subtree of 23Tree's root)
   }
}
```

Performing the analogue of an inorder traversal on a binary tree:

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

```
// Locates the value target in a nonempty 2-3 tree. Returns either the located
// entry or throws an exception if such a node is not found.
findItem(23Tree: TwoThreeTree, target: ItemType): ItemType
{
   if (target is in 23Tree's root node r)
   {  // The data item has been found
      treeItem = the data portion of r
      return treeItem // Success
   }
   else if (r is a leaf)
      throw NotFoundException   // Failure

   // Else search the appropriate subtree
   else if (r has two data items)
   {
      if (target < smaller data item in r)
         return findItem(r's left subtree, target)
      else if (target < larger data item in r)
         return findItem(r's middle subtree, target)
      else
         return findItem(r's right subtree, target)
   }
   else // r has one data item
   {
      if (target < r's data item)
         return findItem(r's left subtree, target)
      else
         return findItem(r's right subtree, target)
   }
}
```

Retrieval operation for a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

- Search of a 2-3 and shortest binary search tree approximately same efficiency
  - A binary search tree with *n* nodes cannot be shorter than $\log_2(n + 1)$
  - A 2-3 tree with n nodes cannot be taller than $\log_2(n + 1)$
  - Node in a 2-3 tree has at most two data items
  - Although you might visit fewer nodes when searching a 2-3 tree (vs a binary tree), you might have to make more comparisons at each node
- Searching 2-3 tree is $O(\log n)$

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

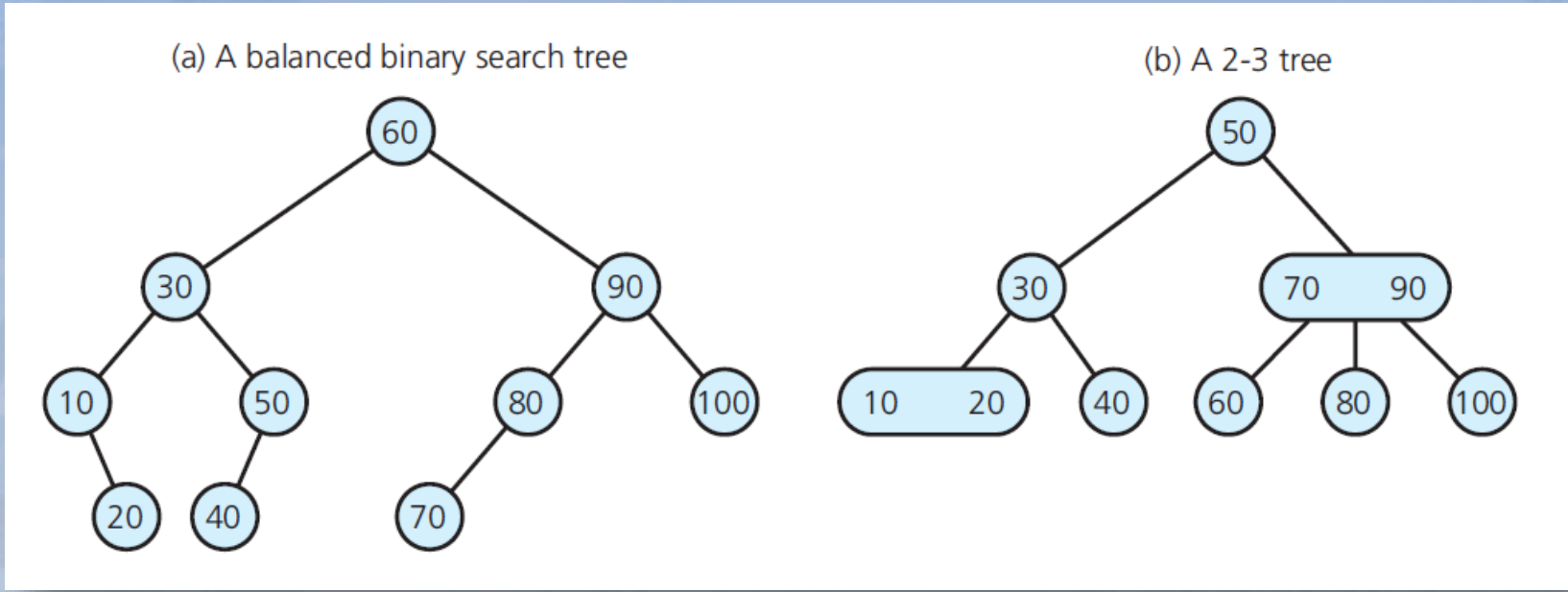


FIGURE 19-9(a) A balanced binary search tree

FIGURE 19-9(b) A 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

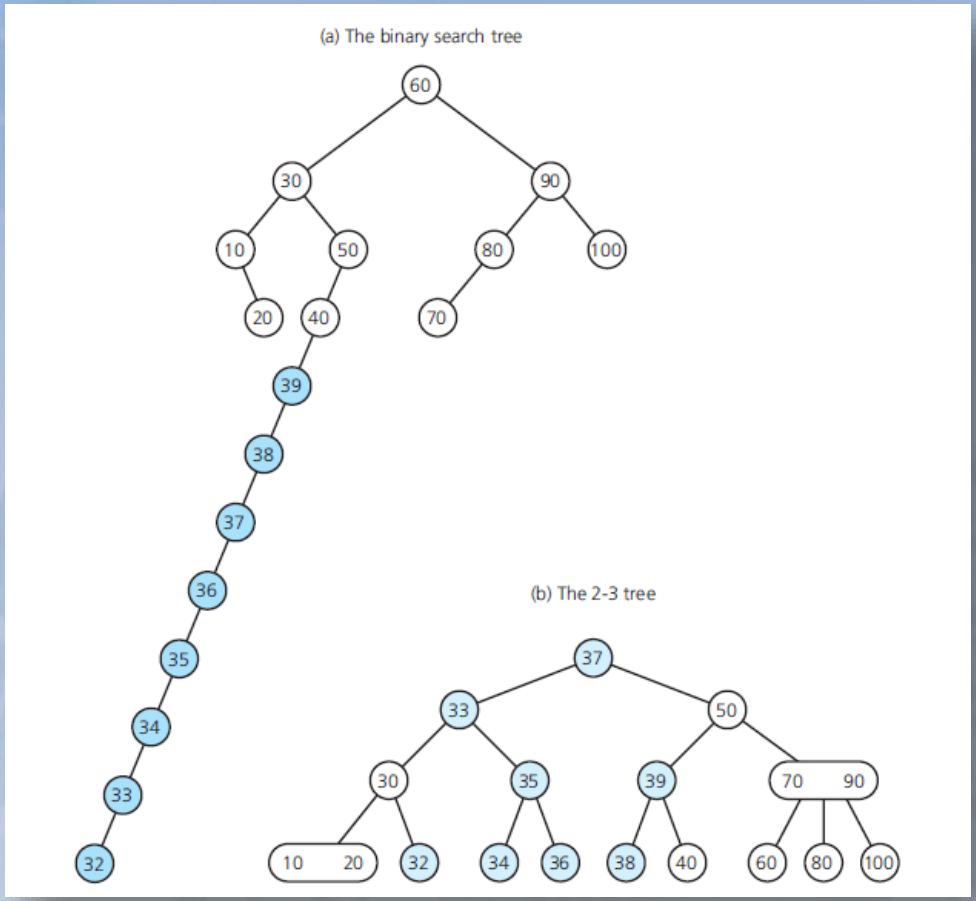
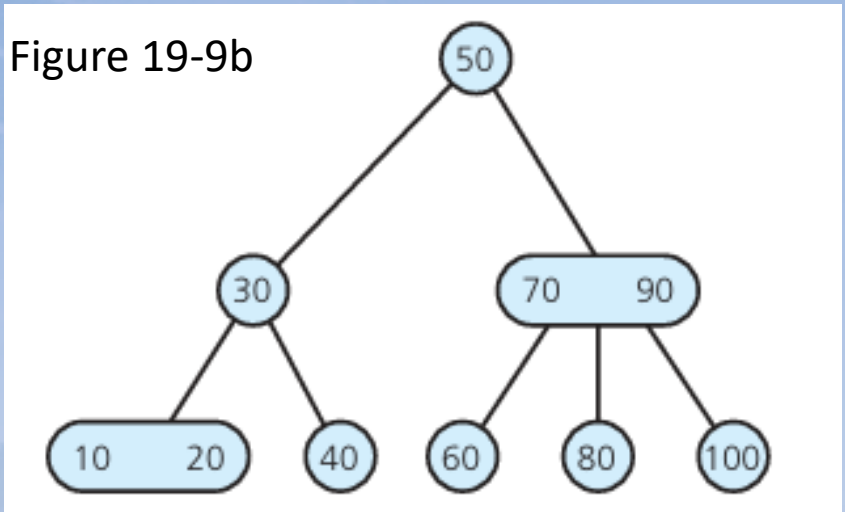


FIGURE 19-10 The trees of Figure 19-9 after adding the values 39 down to 32

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree



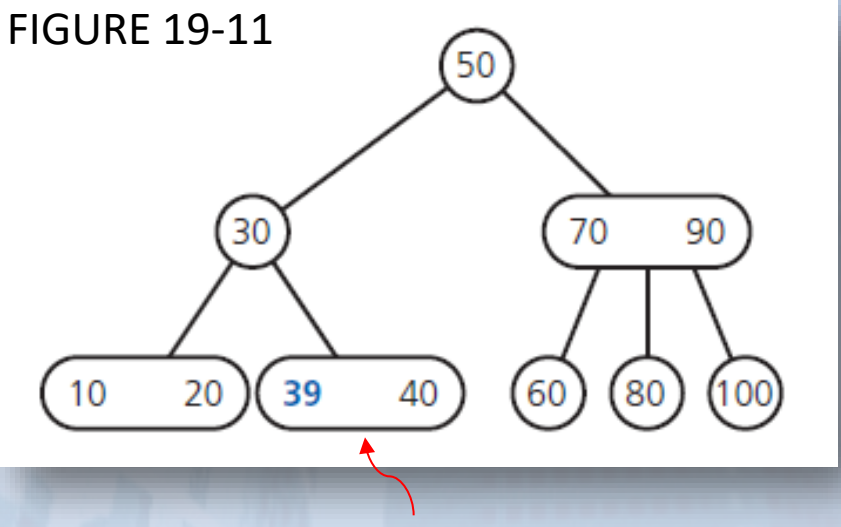


FIGURE 19-11 After inserting 39 into the tree in Figure 19-9b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

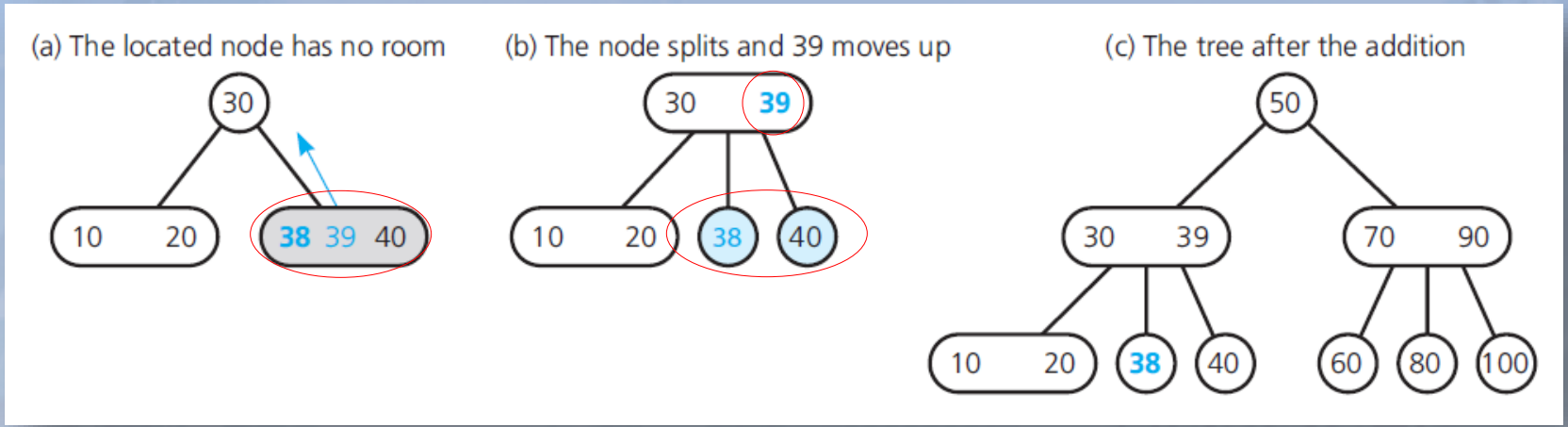
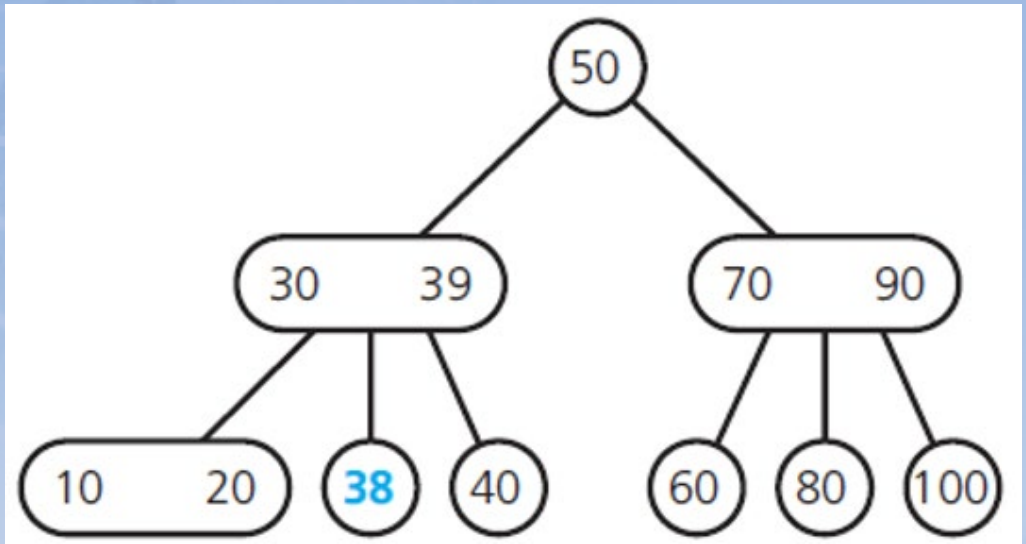


FIGURE 19-12 The steps for adding 38 to the tree in Figure 19-11

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree



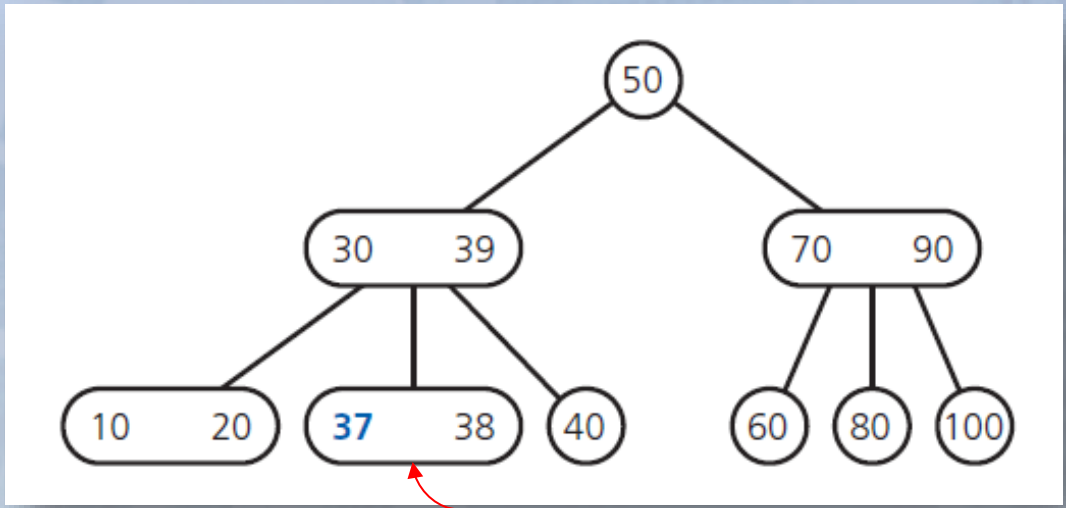
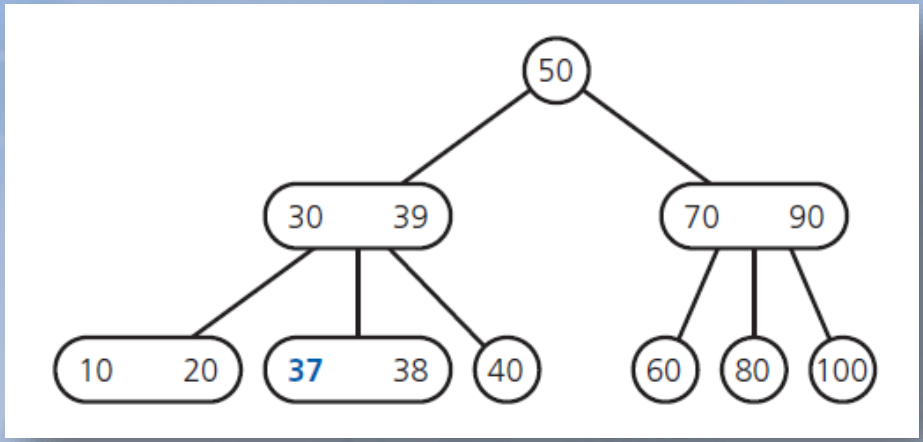


FIGURE 19-13 After adding 37

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree



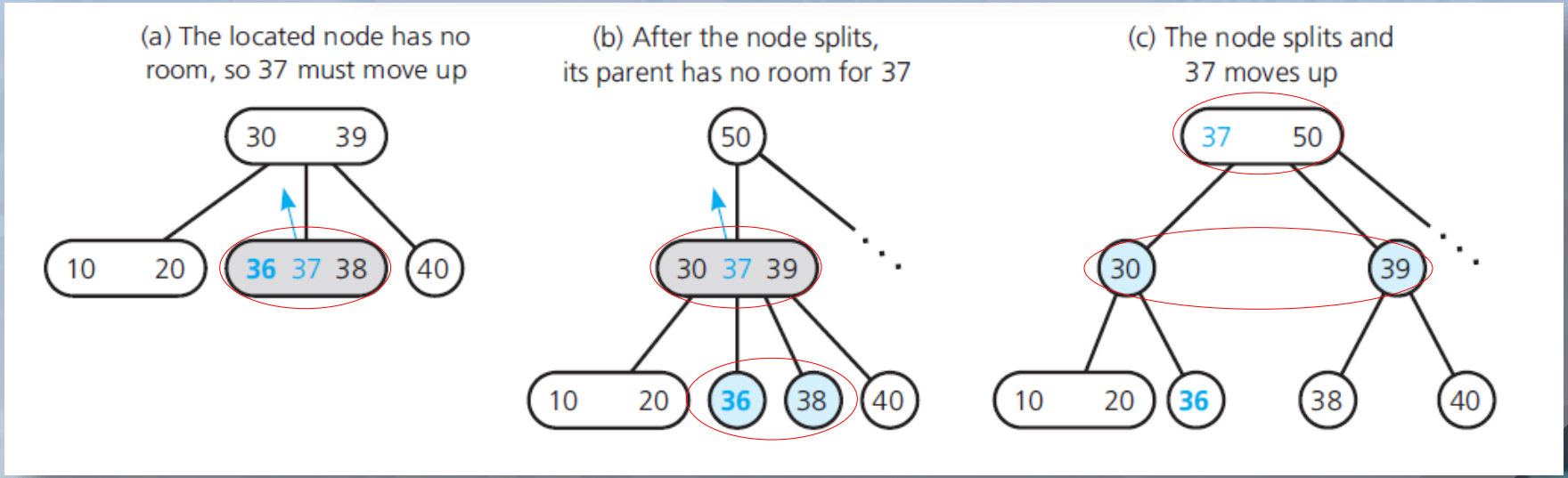


FIGURE 19-14 The steps for adding 36

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

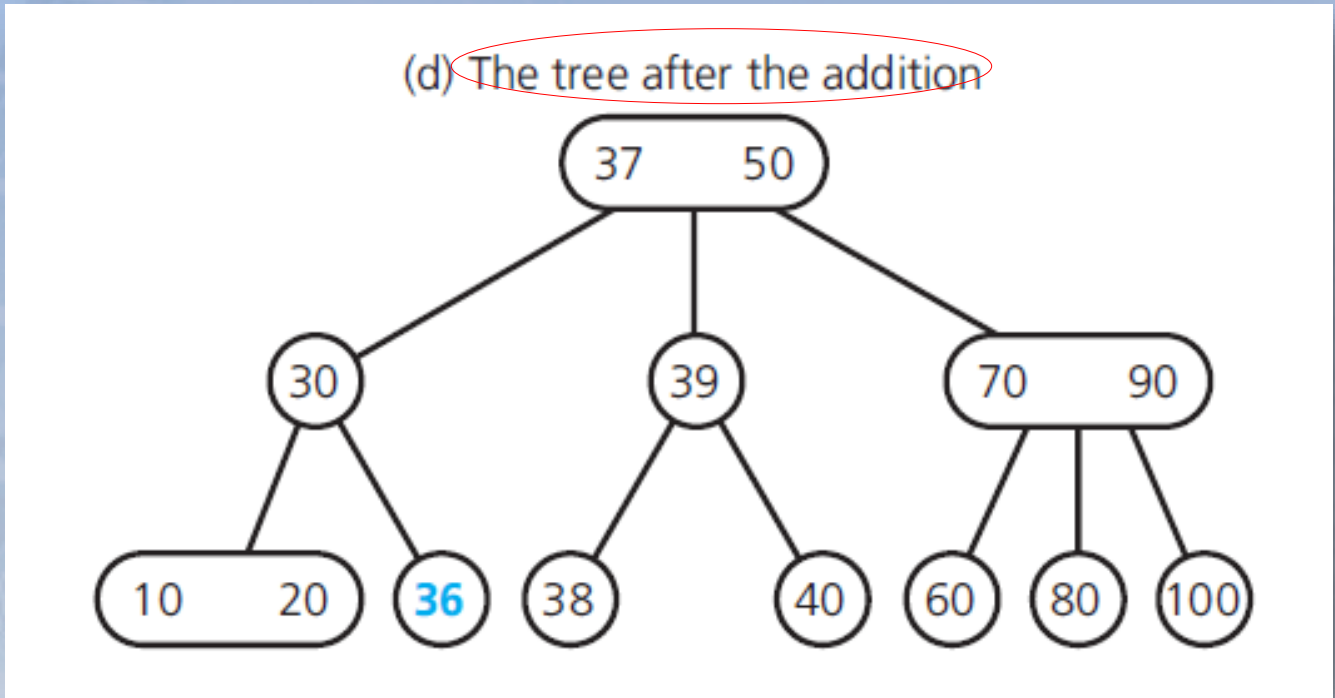


FIGURE 19-14 The steps for adding 36 to the tree in Figure 19-13

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

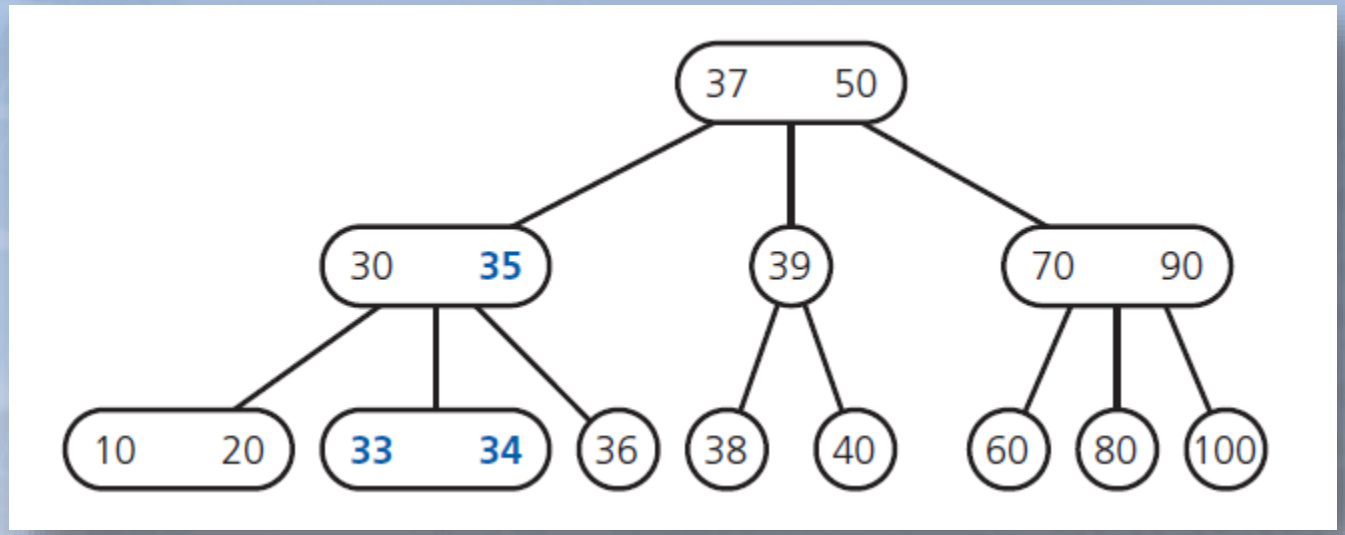


FIGURE 19-15 The tree after the adding 35, 34, and 33 to the tree in Figure 19-14d

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

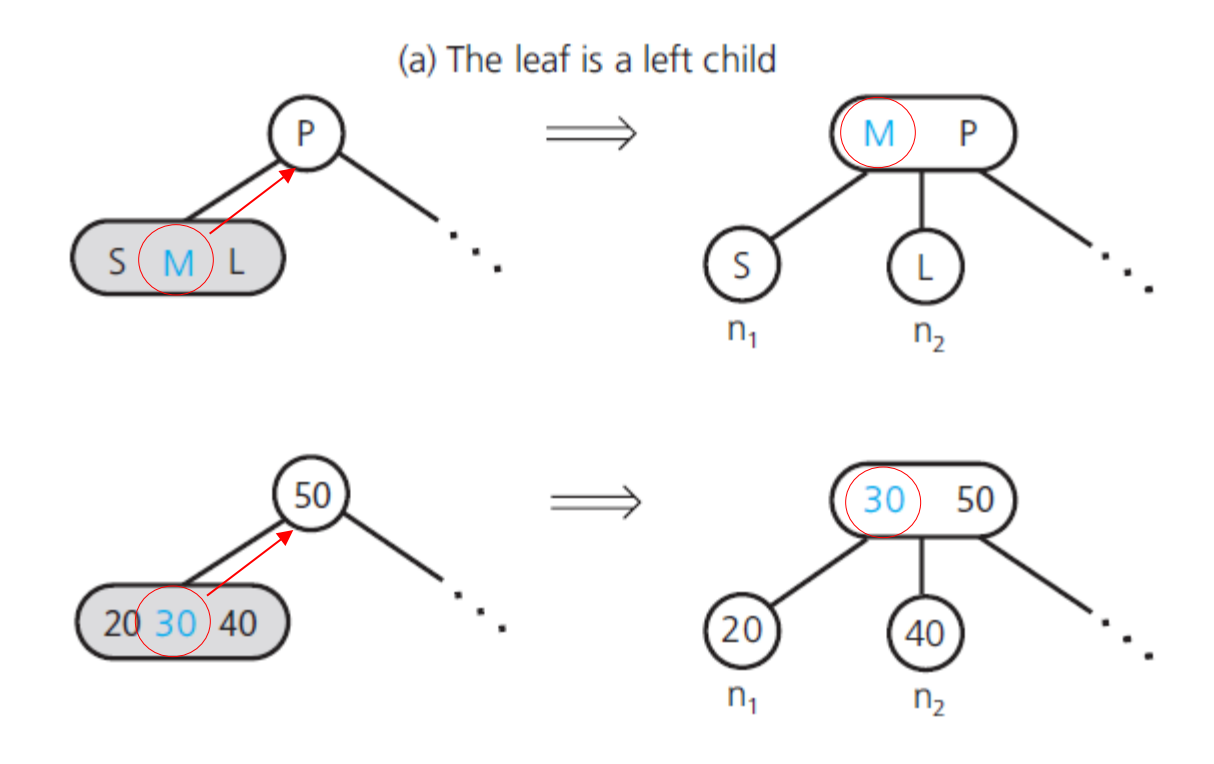


FIGURE 19-16 Splitting a leaf in a 2-3 tree in general and in a specific example

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

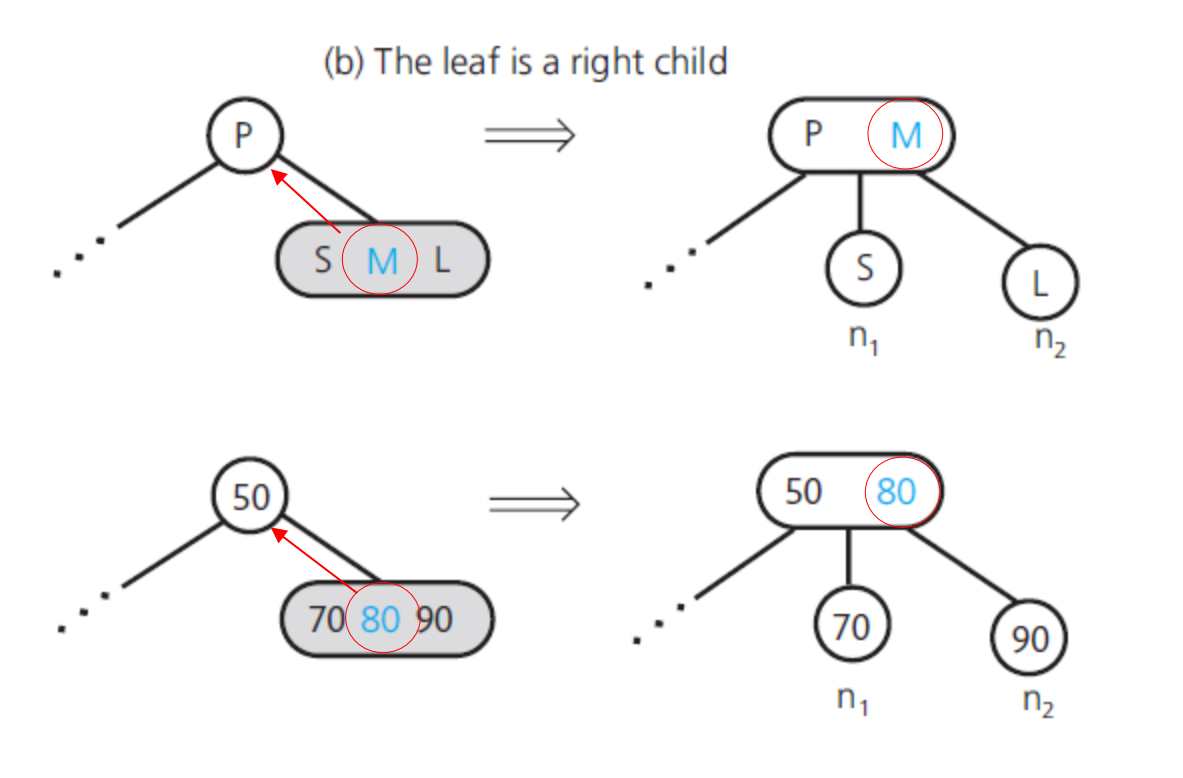


FIGURE 19-16 Splitting a leaf in a 2-3 tree in general and in a specific example

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

Add internal left node by splitting

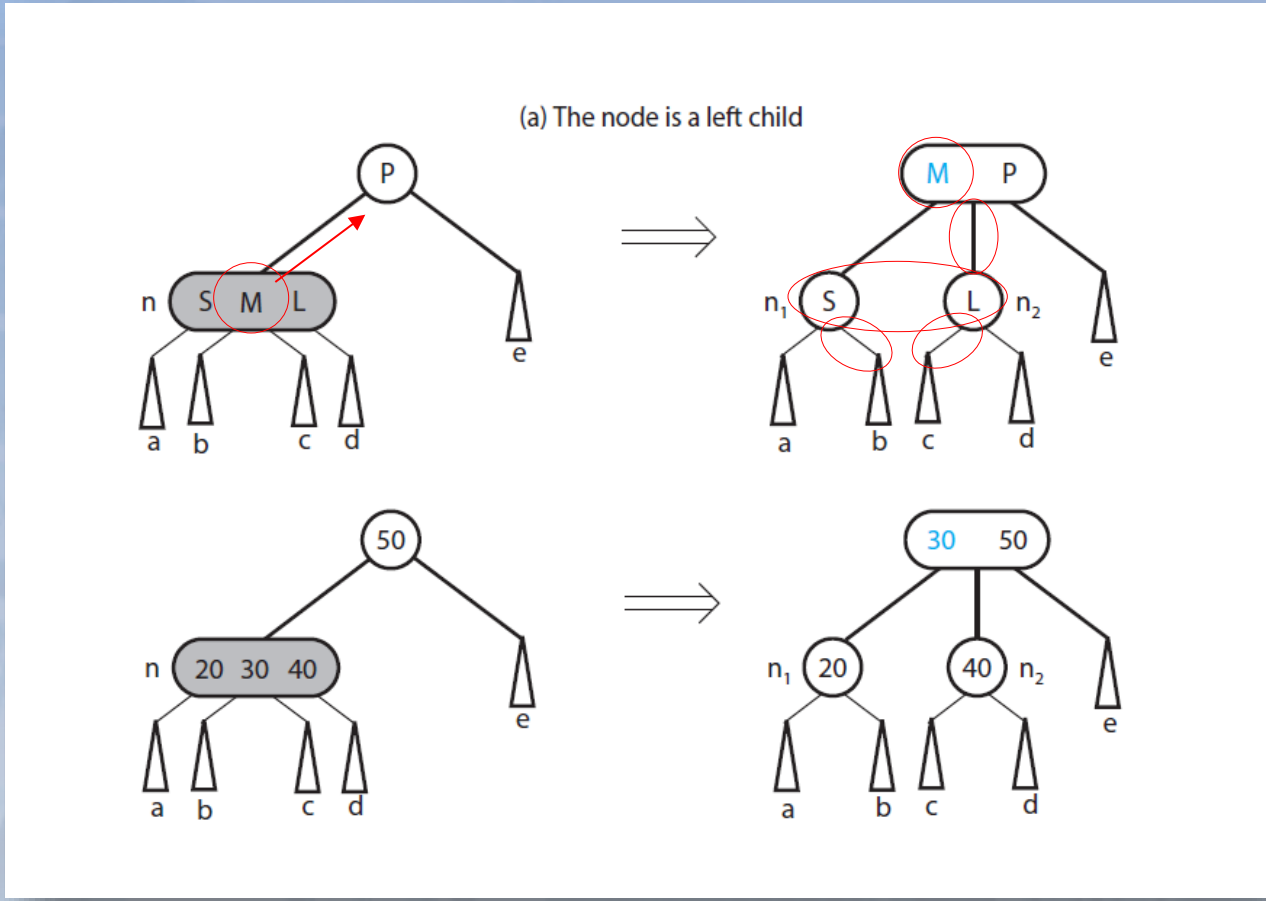


FIGURE 19-17 Splitting an internal node in a 2-3 tree in general and in a specific example

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

Add internal right node by splitting

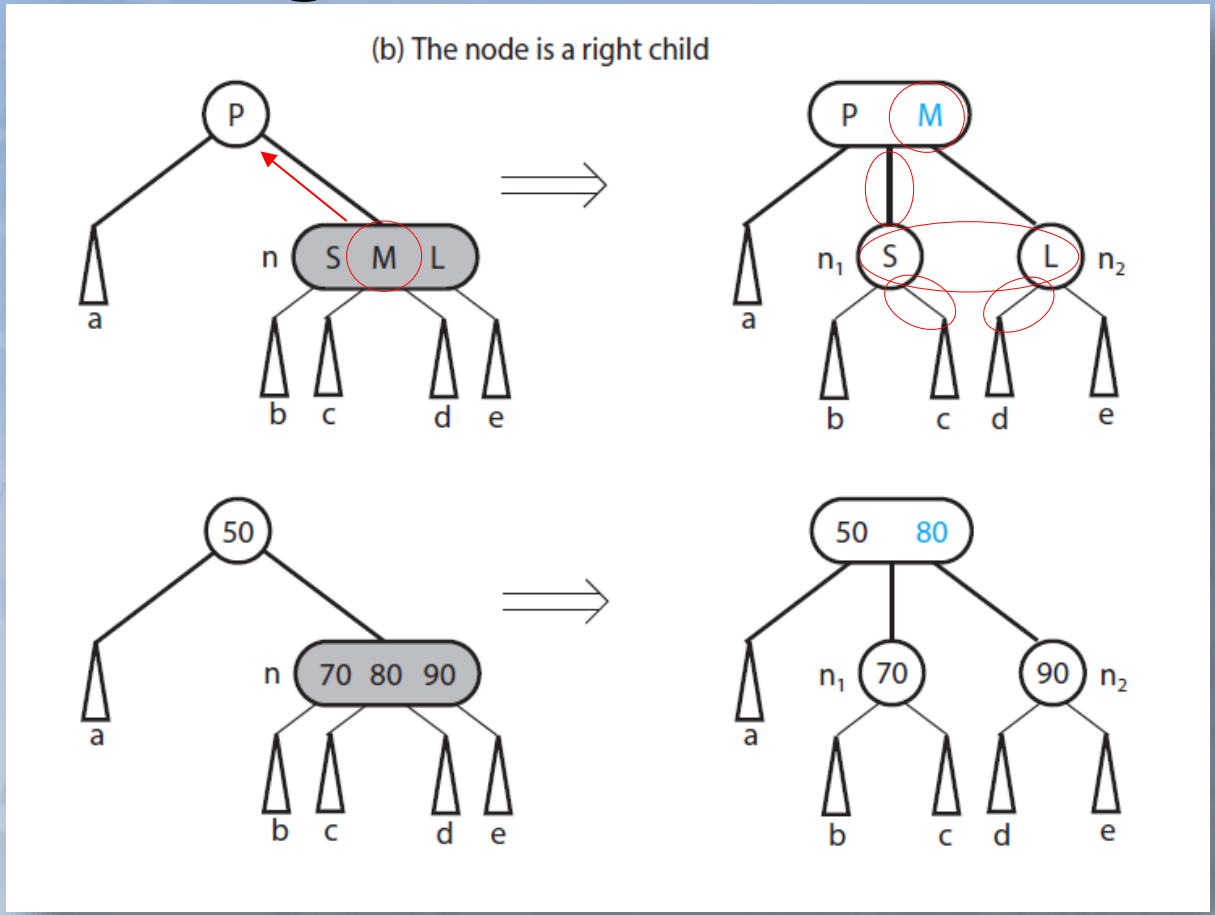


FIGURE 19-17 Splitting an internal node in a 2-3 tree in general and in a specific example

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to a 2-3 Tree

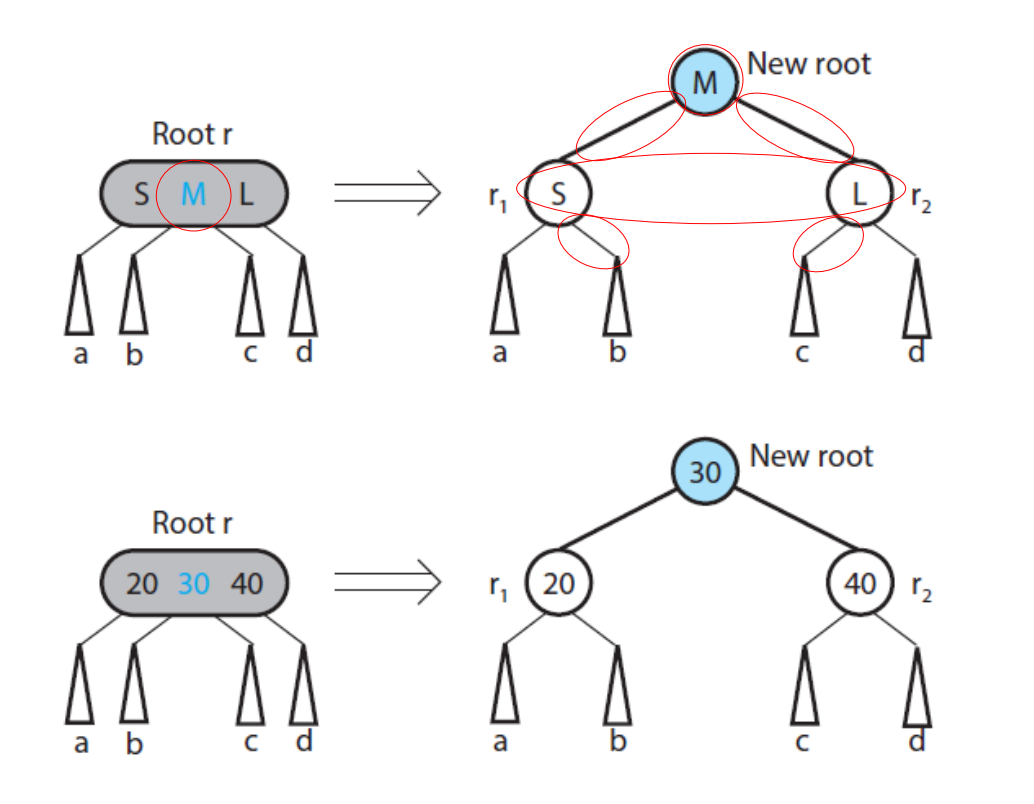


FIGURE 19-18 Splitting the root of a 2-3 tree general and in a specific example

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

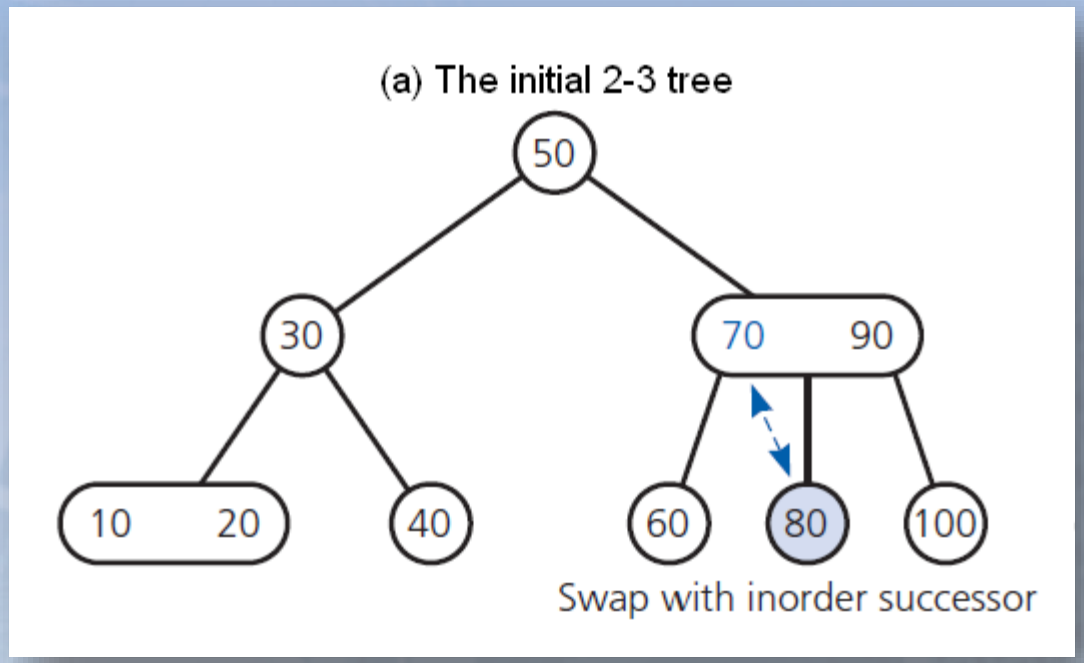


FIGURE 19-19 The steps for removing 70 from the 2-3 tree in Figure 19-9b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

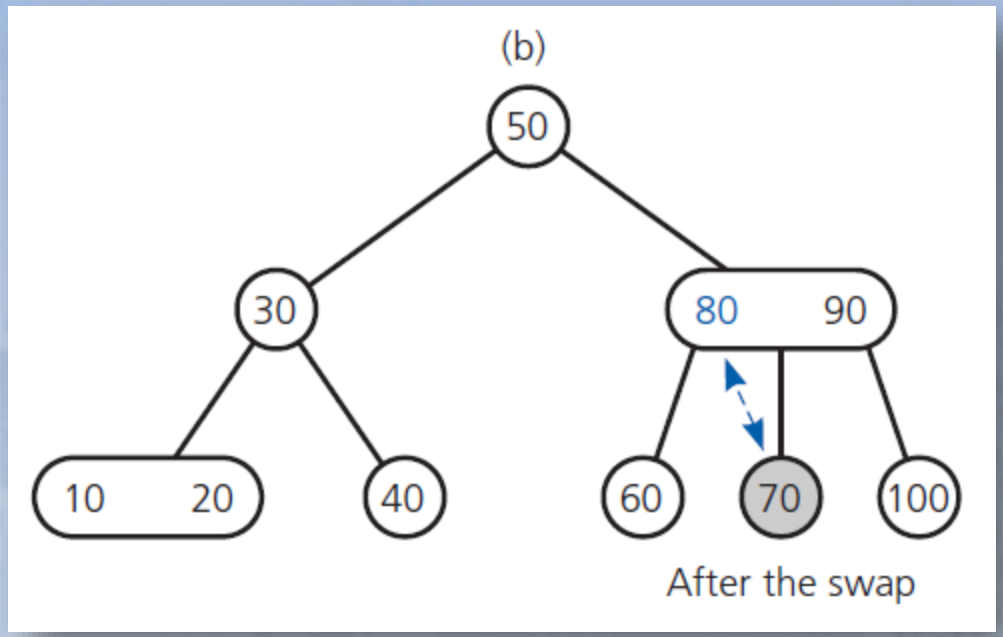
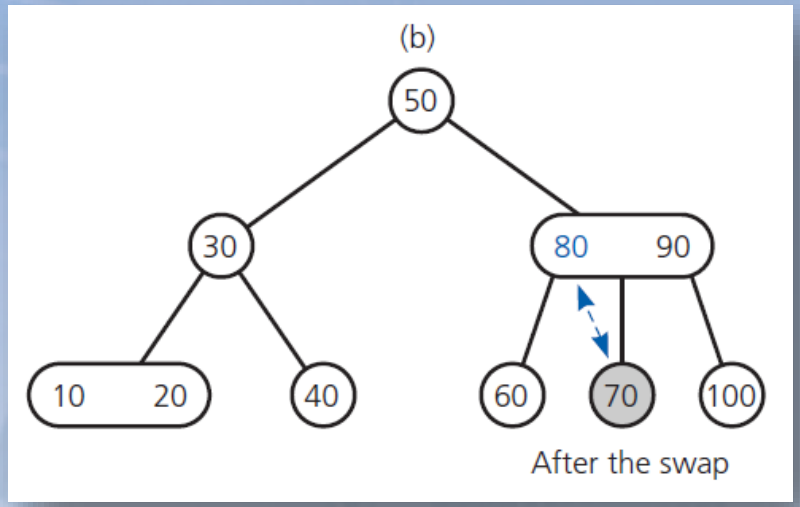
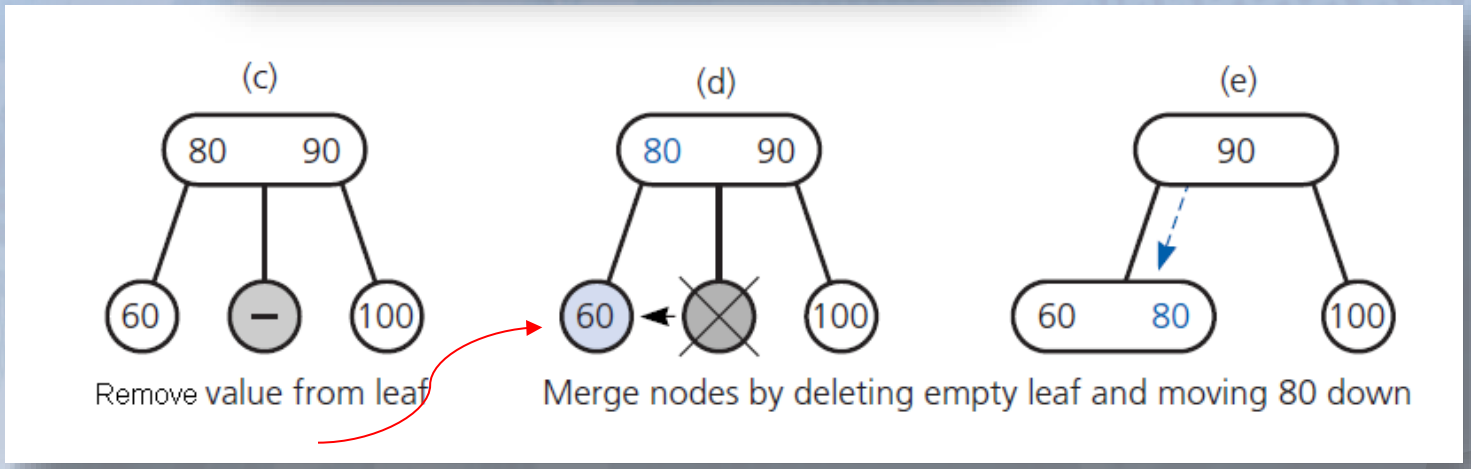


FIGURE 19-19 The steps for removing 70 from the 2-3 tree in Figure 19-9b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree





This situation is not allowed in a 2-3 tree.

FIGURE 19-19 The steps for removing 70 from the 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

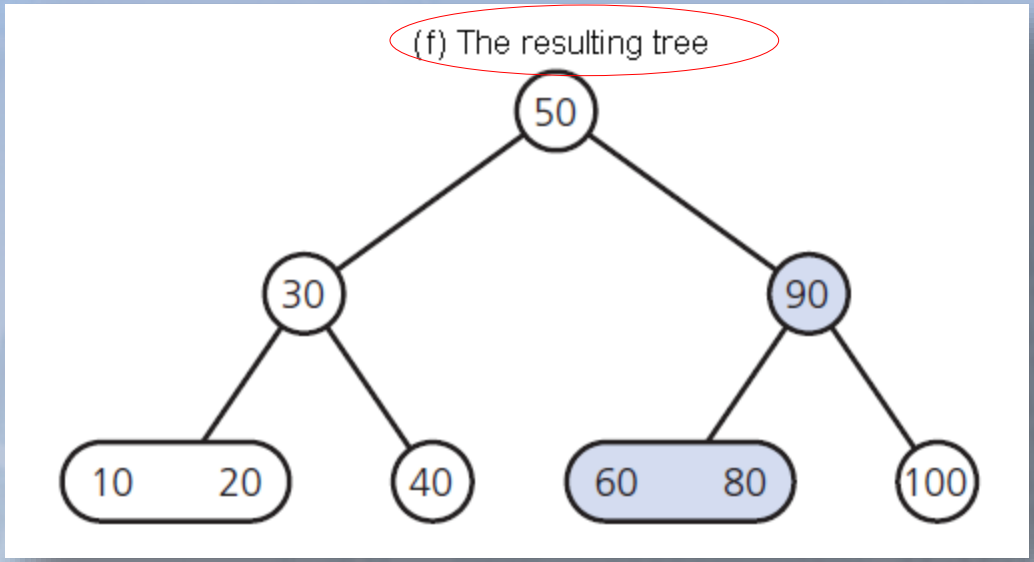
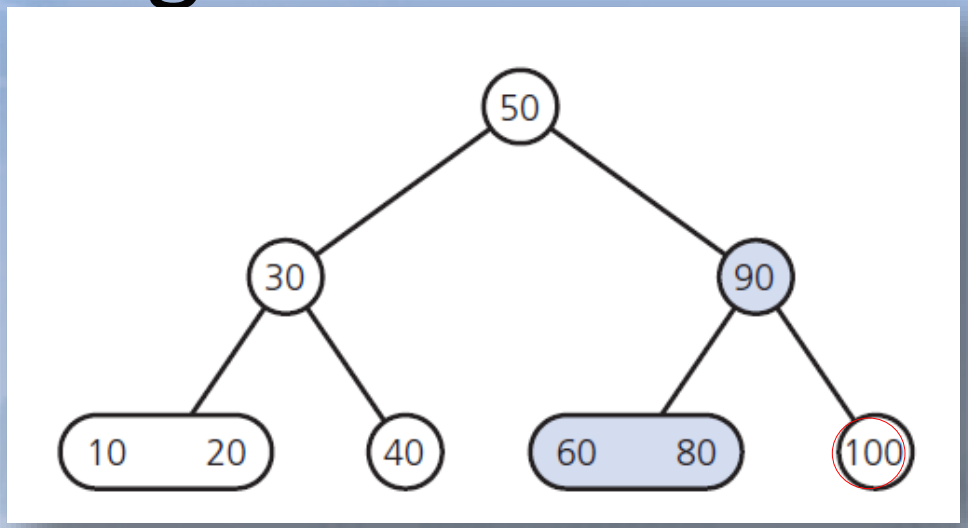


FIGURE 19-19 The steps for removing 70 from the 2-3 tree in Figure 19-9b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree



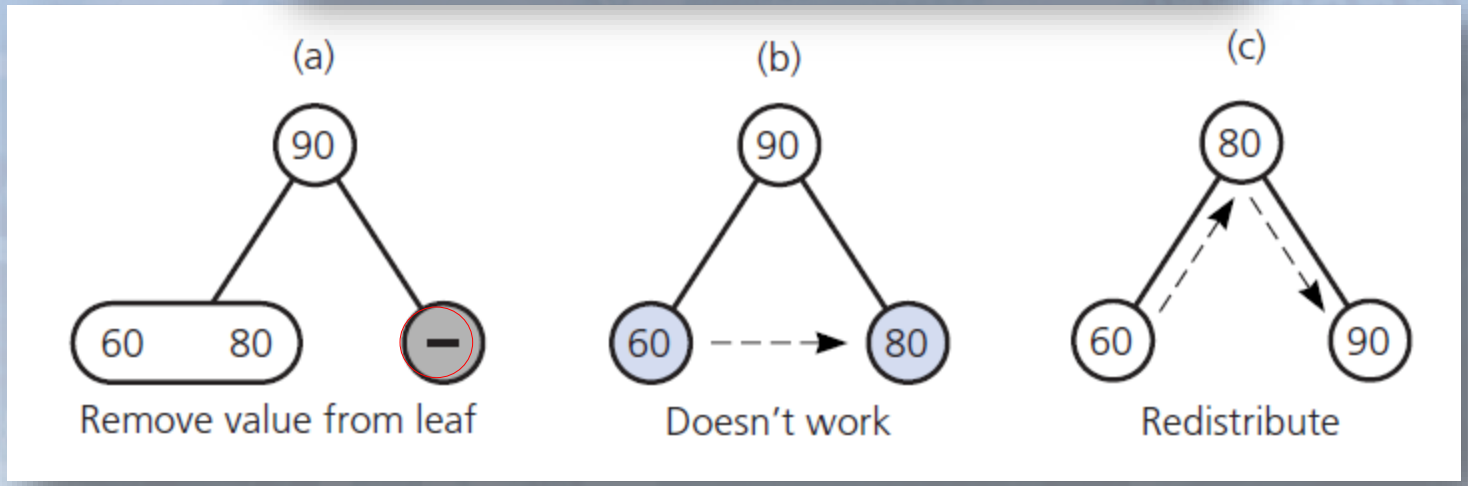


FIGURE 19-20 The steps for removing 100 from the tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

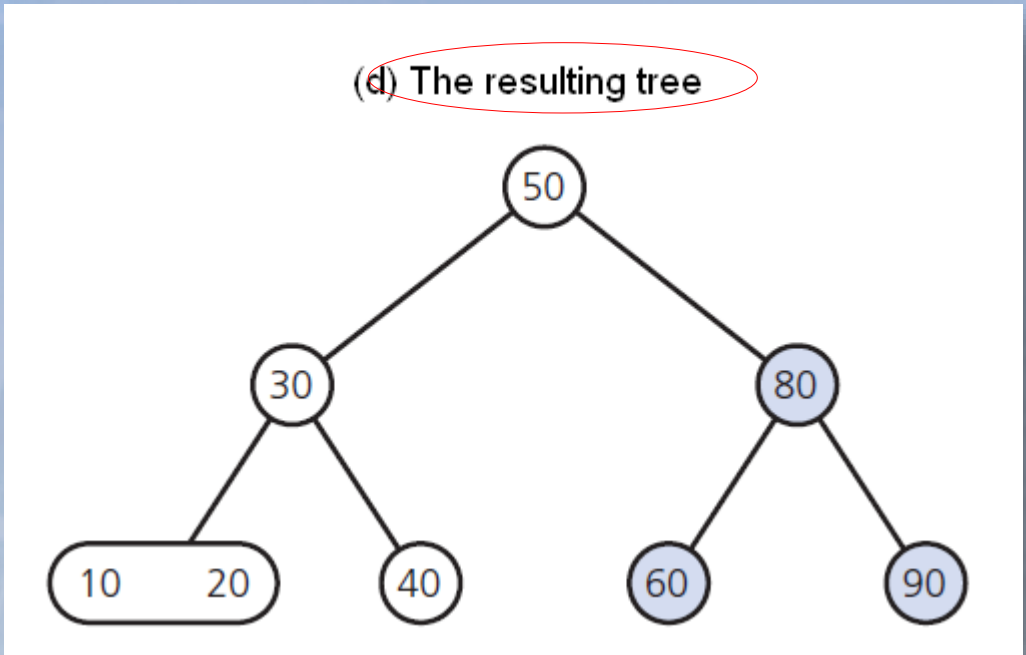
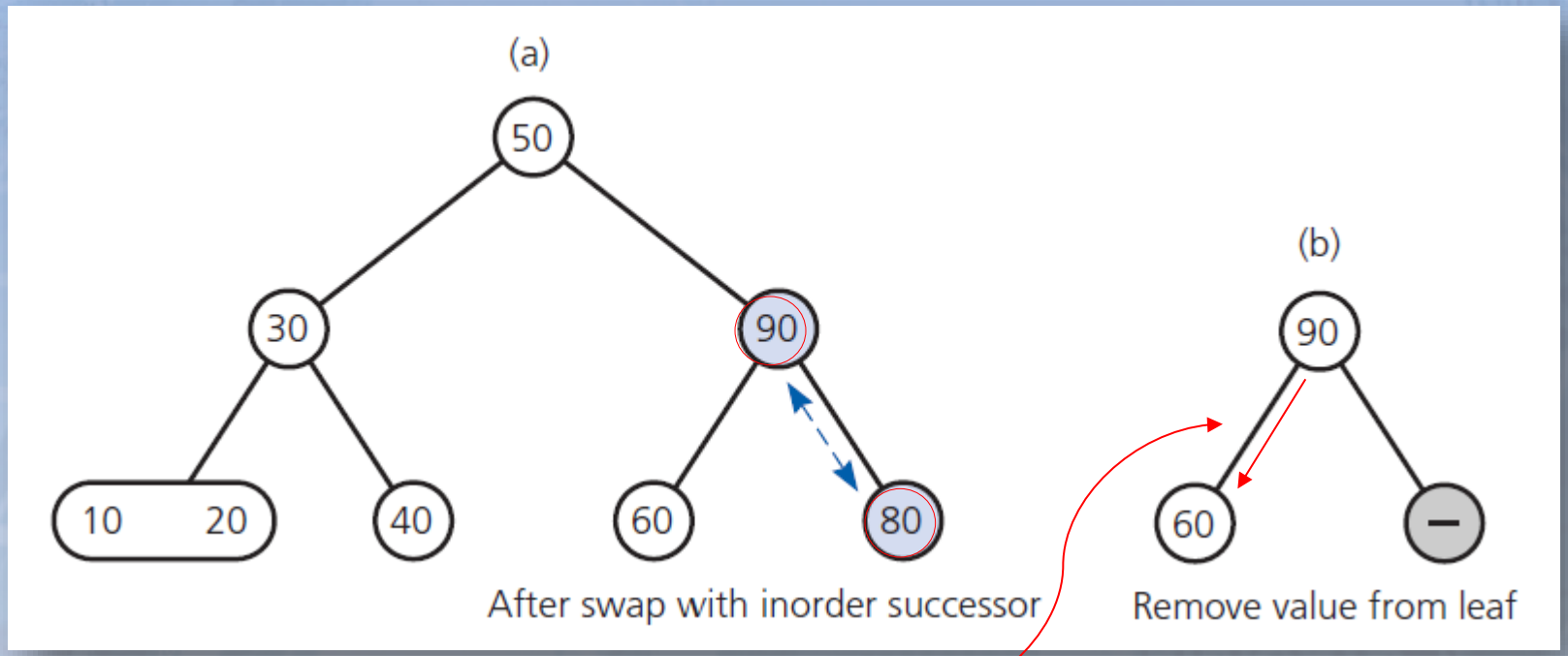


FIGURE 19-20 The steps for removing 100 from the tree in Figure 19-19f;

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree



This situation is not allowed in a 2-3 tree.

Move 90 down to 60 and merge.

FIGURE 19-21 The steps for removing 80 from the 2-3 tree in Figure 19-20d

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

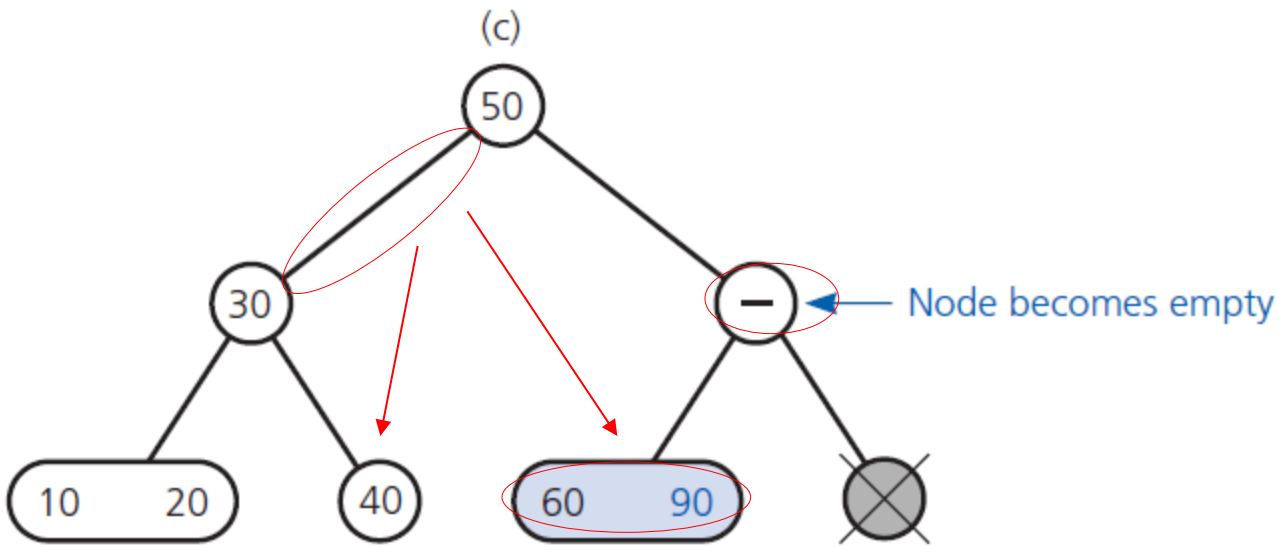
Move 50 down and merge with 30

Point 30-50 middle ptr to 40

Adopt empty leaf's child

Delete empty node

Delete empty root



Move 90 down to 60 and merge; simply repointing 50 to 60-90 does not maintain balance

FIGURE 19-21 The steps for removing 80 from the 2-3 tree in Figure 19-20d

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

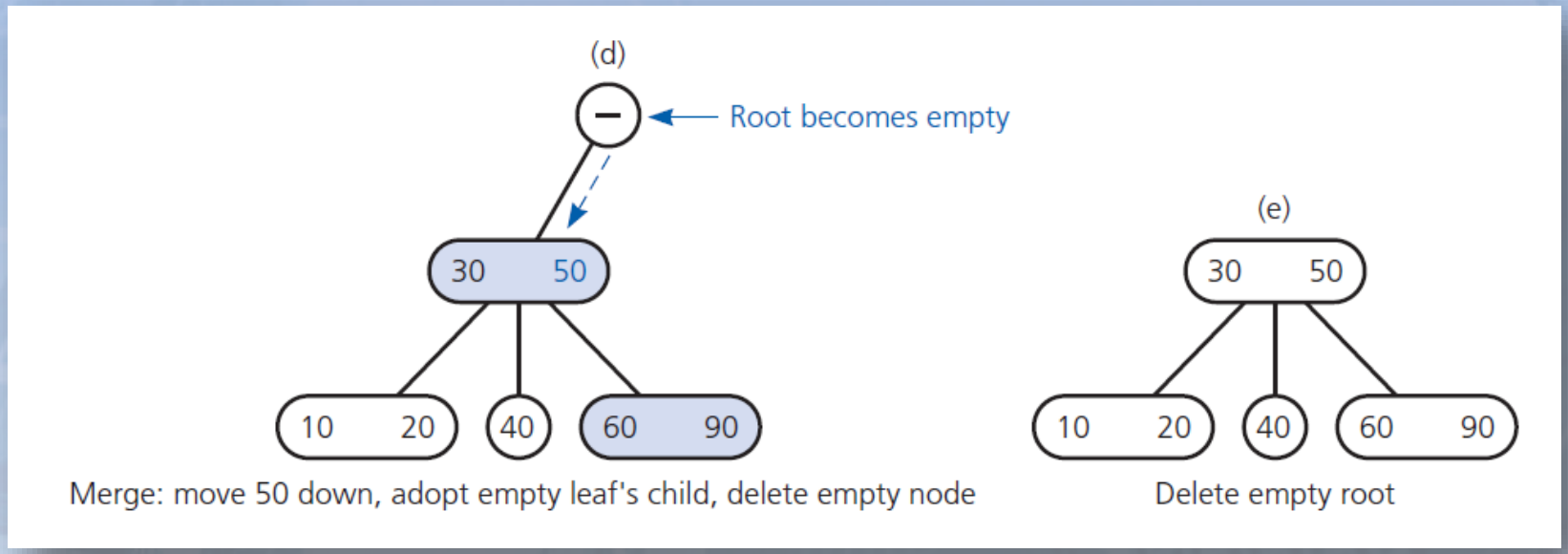


FIGURE 19-21 The steps for removing 80 from the 2-3 tree in Figure 19-20d

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

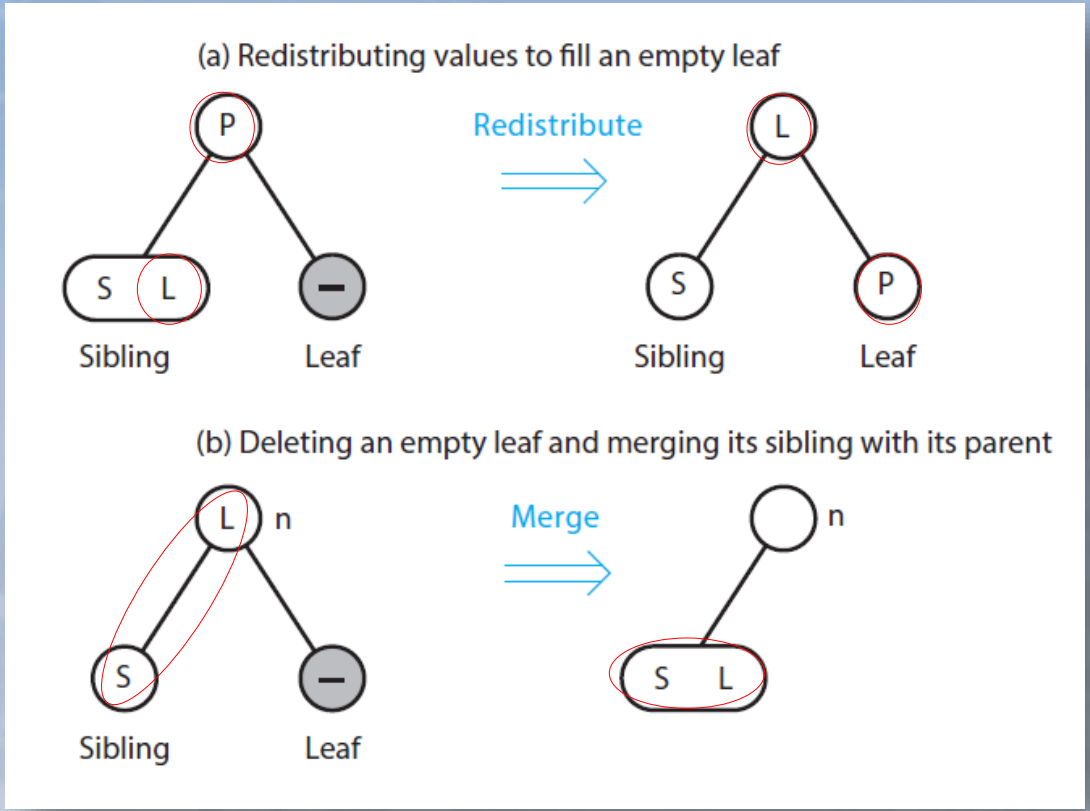


FIGURE 19-23 Possible situations during the removal of a data item from a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

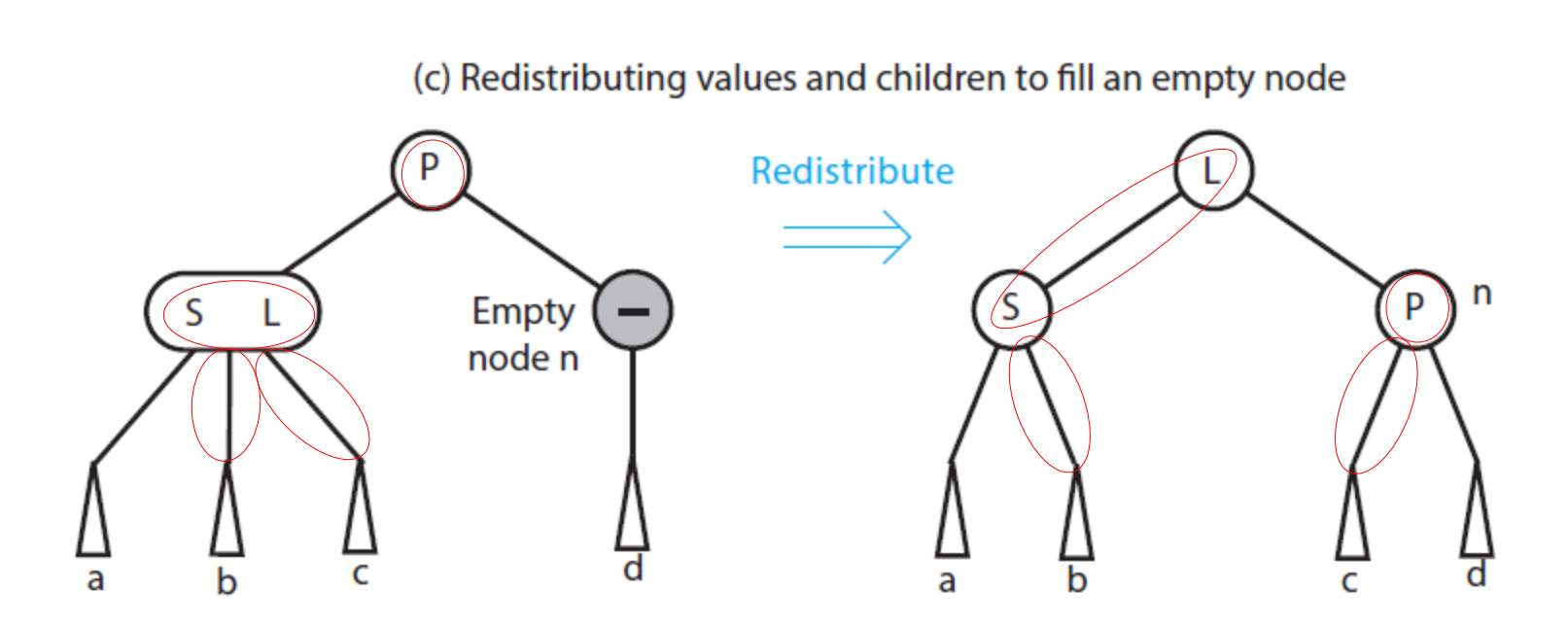


FIGURE 19-23 Possible situations during the removal of a data item from a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

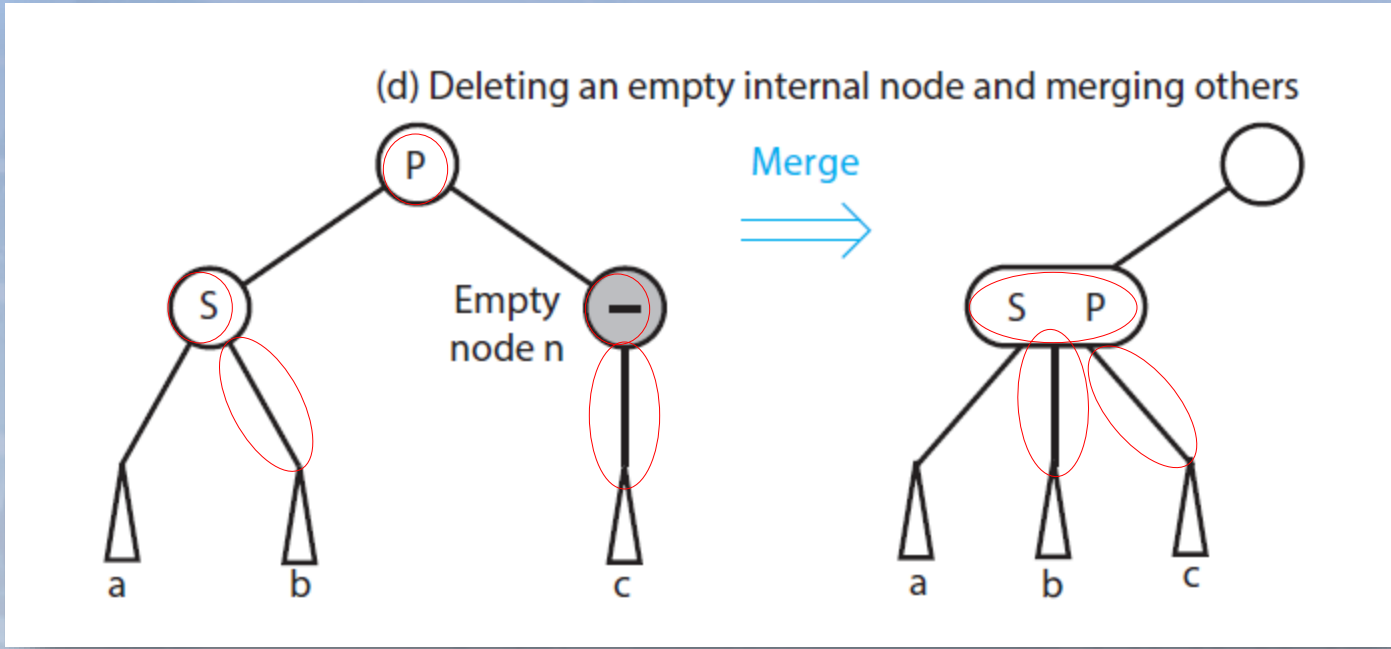


FIGURE 19-23 Possible situations during the removal of a data item from a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

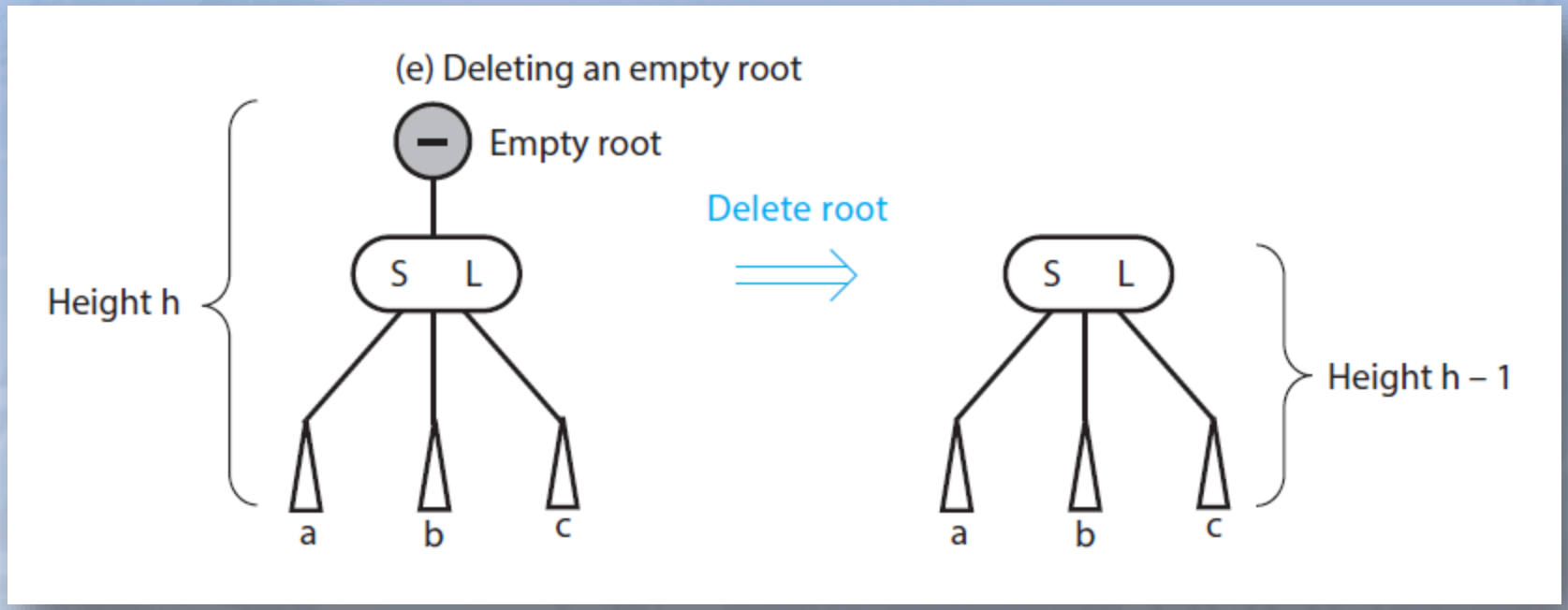


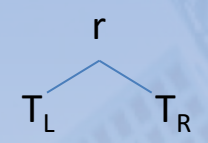
FIGURE 19-23 Possible situations during the removal of a data item from a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

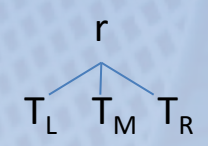
2-3-4 Trees

T is a 2-3-4 tree of height h if one of the following is true:

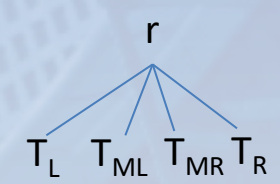
1. T is empty, in which case h is 0.
2. T is of the form
   a. r is a node that contains one data item
   b. T_L and T_R are both 2-3-4 trees, each of height h-1
   c. $r > T_L$ and $r < T_R$



3. T is of the form
   a. r is a node that contains two data items;
   b. T_L, T_M and T_R are both 2-3-4 trees, each of height h-1
   c. smaller $r > T_L$ and $r < T_M$; larger $r > T_M$ and $< T_R$

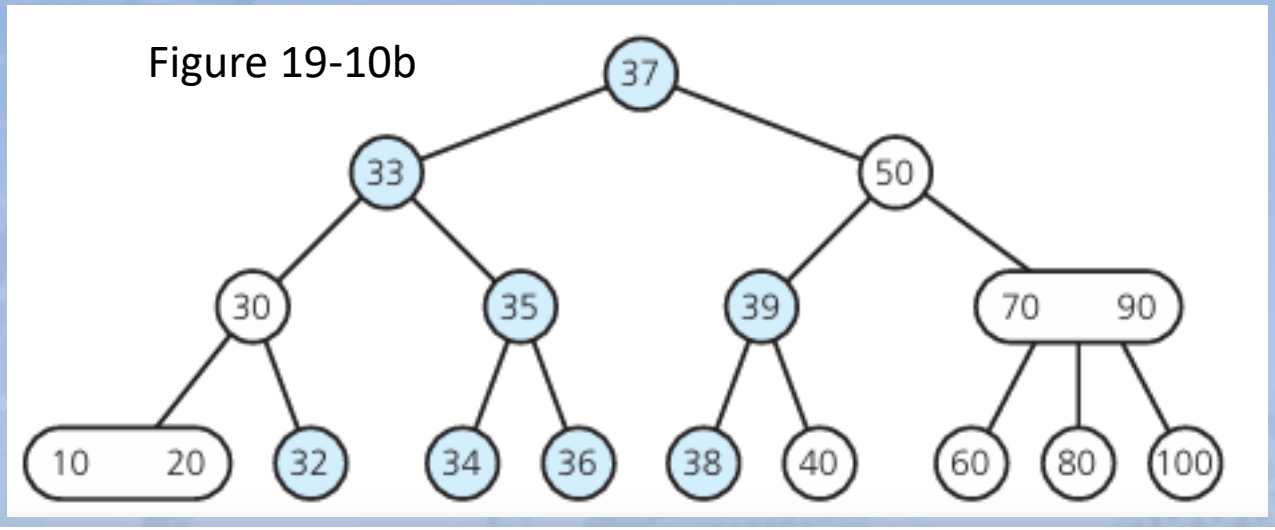


4. T is of the form
   a. r is a node that contains three data items;
   b. T_L, T_{ML}, T_{MR} and T_R are both 2-3-4 trees, each of height h-1
   c. smallest $r > T_L$ and smallest $r < T_{ML}$
   d. middle $r > T_{ML}$ and middle $r < T_{MR}$
   e. largest $r > T_{MR}$ and largest $r < T_R$



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3-4 Trees



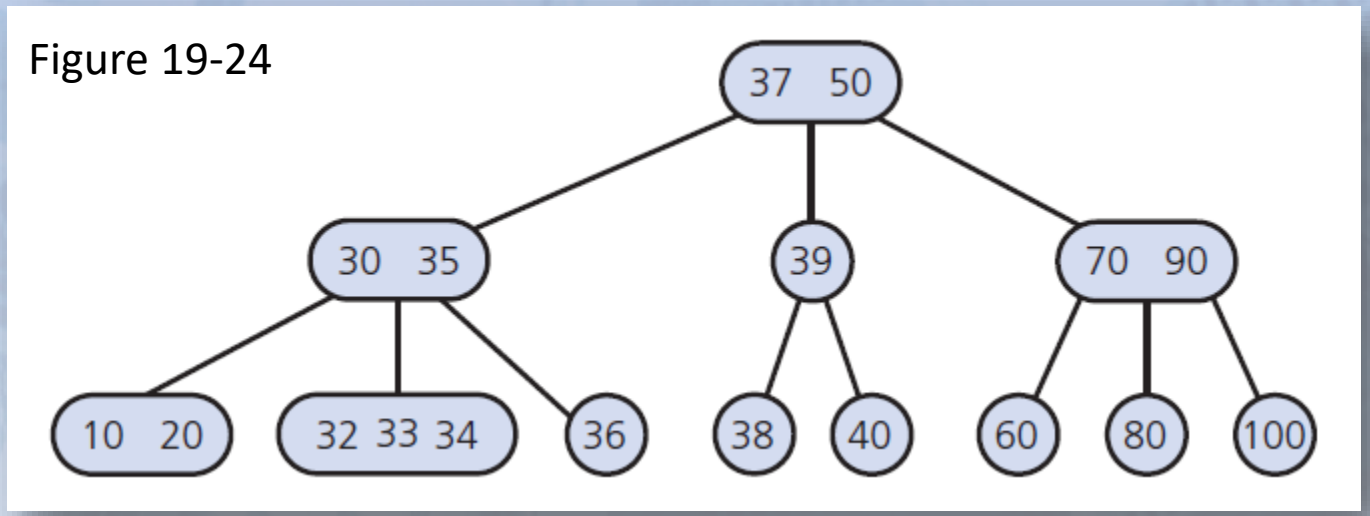


FIGURE 19-24 A 2-3-4 tree with the same data items as the 2-3 tree in Figure 19-10b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

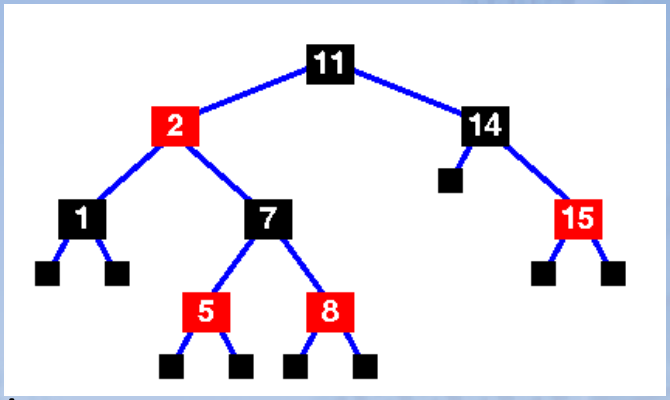
2-3-4 Trees

- Searching and traversing
  - Simple extensions of corresponding algorithms for a 2-3 tree
- Adding data
  - Like addition algorithm for 2-3 tree
  - Splits node by moving one data item up to parent node

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black Trees

- [1]Invented in 1978 Leonidas Guibas and Robert Sedgewick
- Self balancing binary search tree; O(log n) search/add/delete
- [2]Red-black tree properties
  - Each node is either red or black
  - Root node is always black in color
  - Maintain balance during add/remove operations
  - Nodes that have no child are considered the internal nodes and these nodes are connected to the NIL nodes that are always black in color. The NIL nodes are the leaf nodes in the Red-Black tree.
  - If the node is Red, then its children should be in Black color.
  - Every path from a node to any of its descendant's NIL node should have same number of black nodes.



[3]A basic red-black tree

[1]https://en.wikipedia.org/wiki/Red%E2%80%93black_tree#:~:text=AVL%20trees%20can%20be%20colored,trees%20are%20more%20rigidly%20balanced.
[2]https://www.javatpoint.com/red-black-tree
[3]https://www.eecs.umich.edu/courses/eecs380/ALG/red_black.html

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3-4 vs Red-Black Trees

- A 2-3-4 tree:
  - Balanced
  - Add/remove operations use one pass from root to leaf
  - Requires more storage than binary search tree
- Red-black tree
  - Special binary tree which represents 2-3-4 tree
  - Advantages of a 2-3-4 tree but requires less storage
- Represent each 3 node and 4 node in a 2-3-4 tree as equivalent binary search tree
  - All child pointers in original 2-3-4 tree are black pointers
  - Red pointers link 2-nodes that now contain values that were in a 3-node or a 4-node.
- Worst case search, insert and delete O(log n)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black Trees

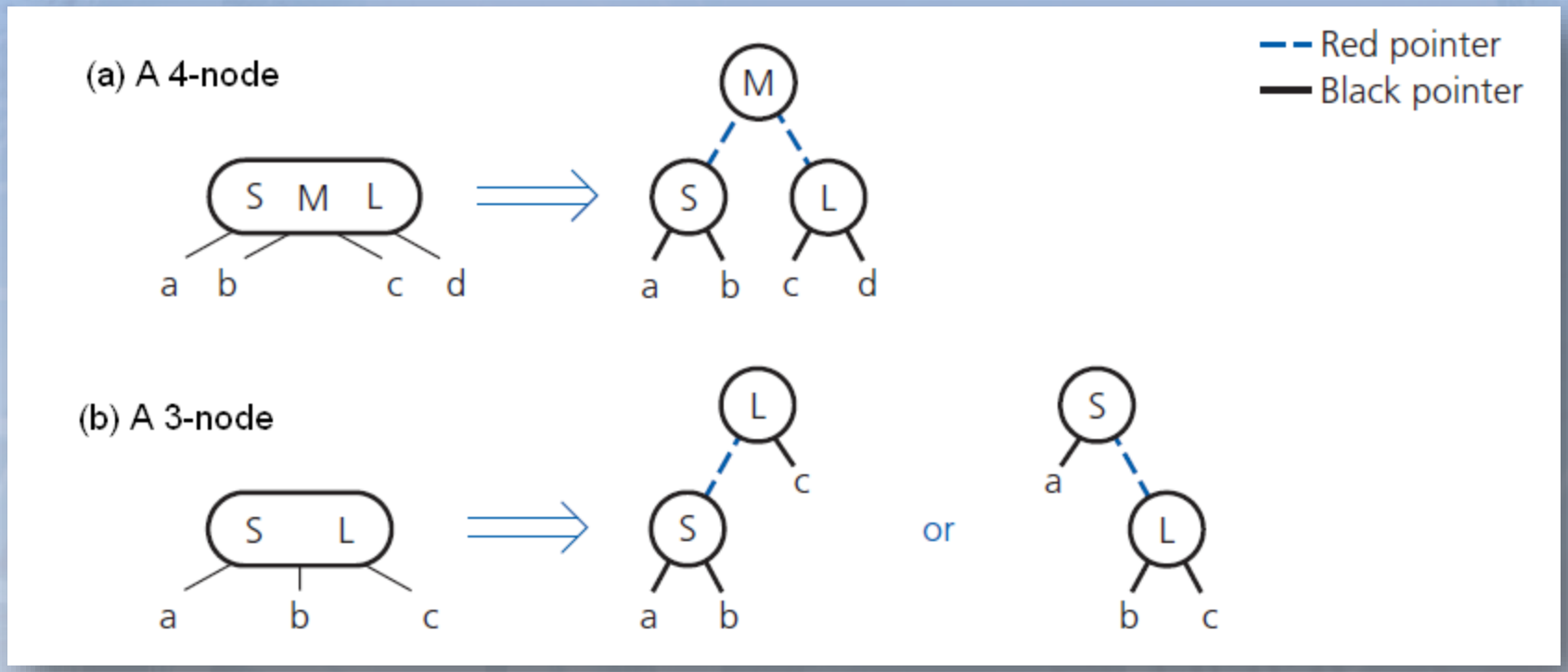


FIGURE 19-35 Red-black representation s of a 4-node and a 3-node

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black Trees

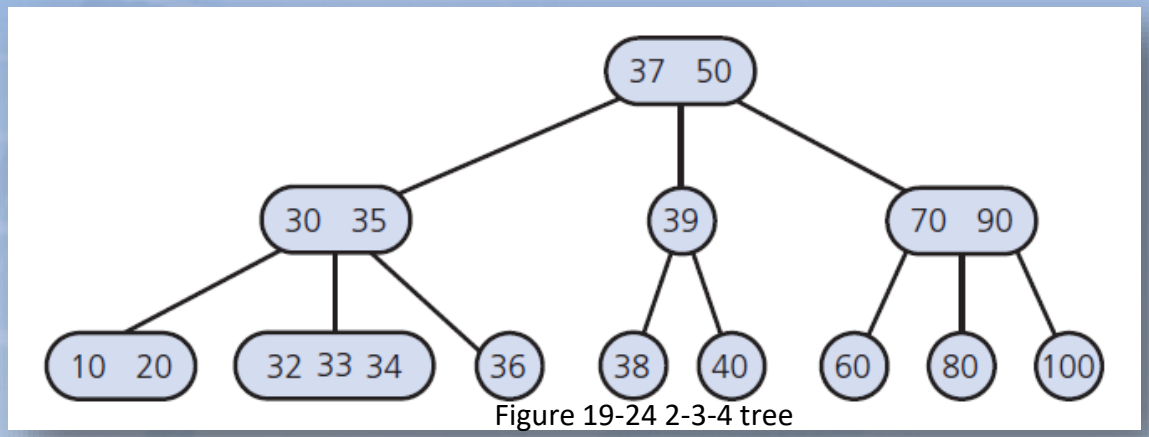


Figure 19-24 2-3-4 tree

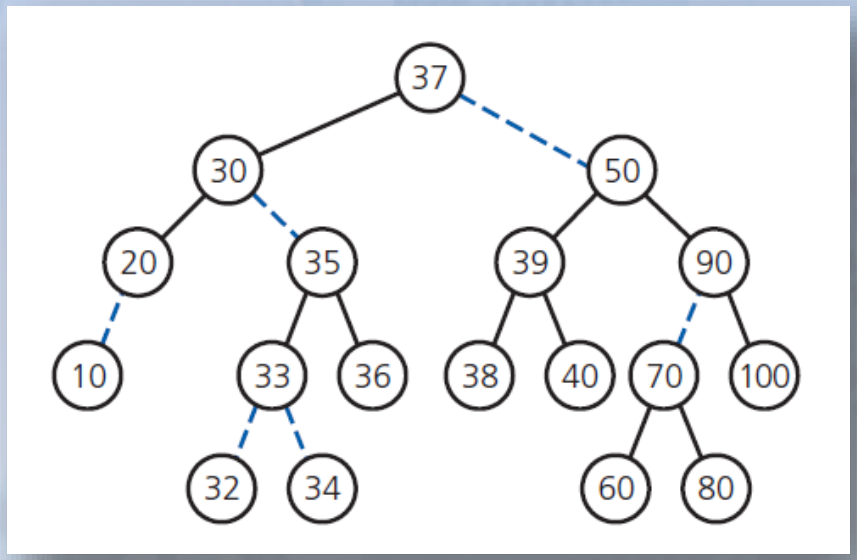


FIGURE 19-36 A red-black tree that represents the 2-3-4 tree in Figure 19-24

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching and Traversing a Red-Black Tree

- A red-black tree is a binary search tree
- Thus, search and traversal
  - Use algorithms for binary search tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

- Red-black tree represents a 2-3-4 tree
  - Simply adjust 2-3-4 addition algorithms
  - Accommodate red-black representation
- Splitting equivalent of a 4-node requires simple color changes
  - Pointer changes called rotations result in a shorter tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black vs AVL Trees

	Red-Black	AVL
[1, 2]Height	2 * log (n + 1)	1.44 * log n
[3]Add/delete operations	faster add/delete operations; fewer rotations are due to relatively balancing	complex add/delete operations; more rotations are due to strict balancing
[3]Storage	requires only 1 bit of information per node	store balance factors or heights with each node thus requiring storage for an integer per node
[3]Uses	used in most of the language libraries like map, multimap, multiset in C++, etc	used in databases where faster retrievals are required
[3]Balancing	Take less processing for balancing i.e.; maximum two rotation required	Take more processing for balancing

[1]https://pages.cs.wisc.edu/~siff/CS367/Notes/rbts.html#:~:text=It%20can%20be%20proven%20that,O(lg(n)).
[2]https://pages.cs.wisc.edu/~ealexand/cs367/NOTES/AVL-Trees/index.html
[3]https://www.geeksforgeeks.org/red-black-tree-vs-avl-tree/

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Chapter 19

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 19

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3 Tree

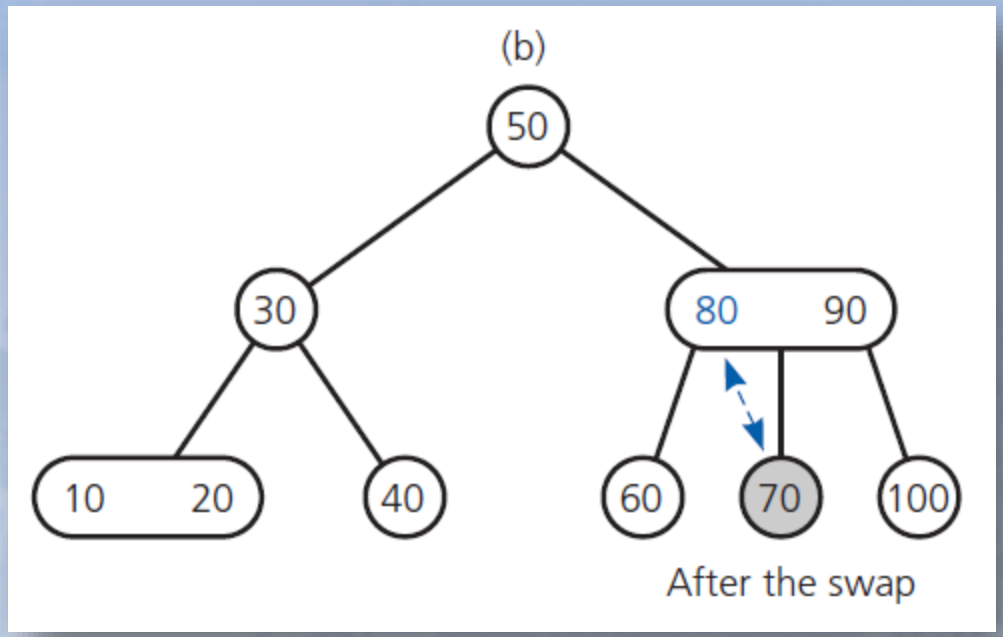


FIGURE 19-19 The steps for removing 70 from the 2-3 tree in Figure 19-9b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3-4 Trees

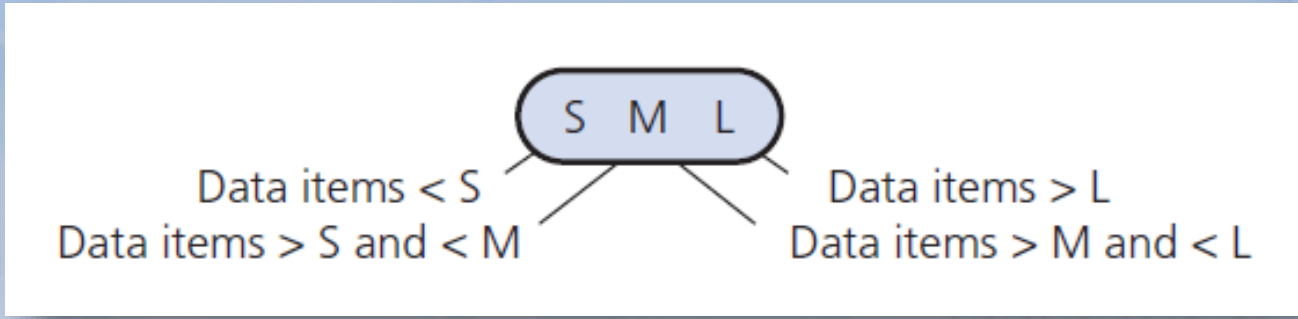


FIGURE 19-25 A 4-node in a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

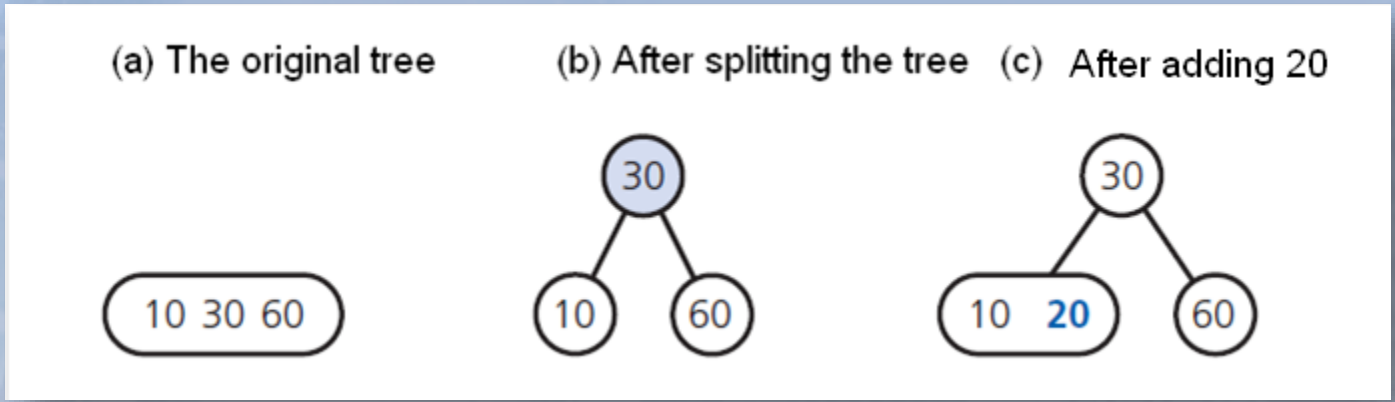


FIGURE 19-26 Adding 20 to a one-node 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

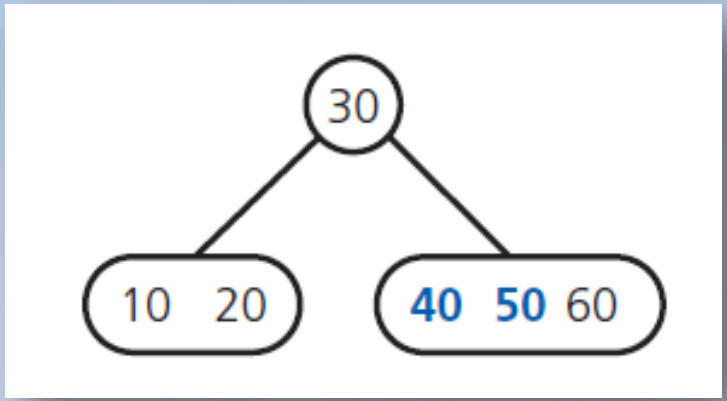


FIGURE 19-27 After adding 50 and 40 to the tree in Figure 19-26c

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

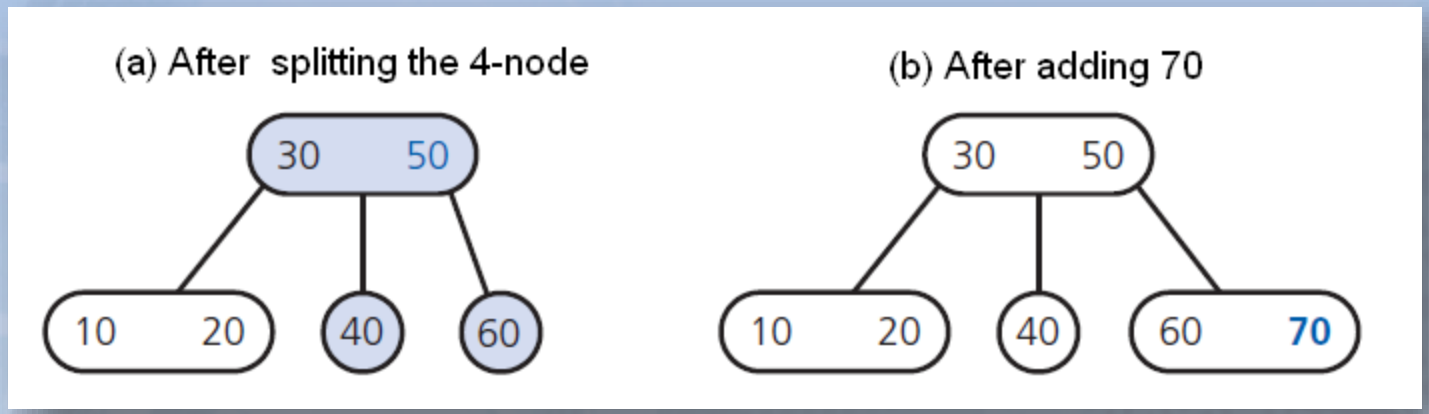


FIGURE 19-28 The steps for adding 70 to the tree in Figure 19-27

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

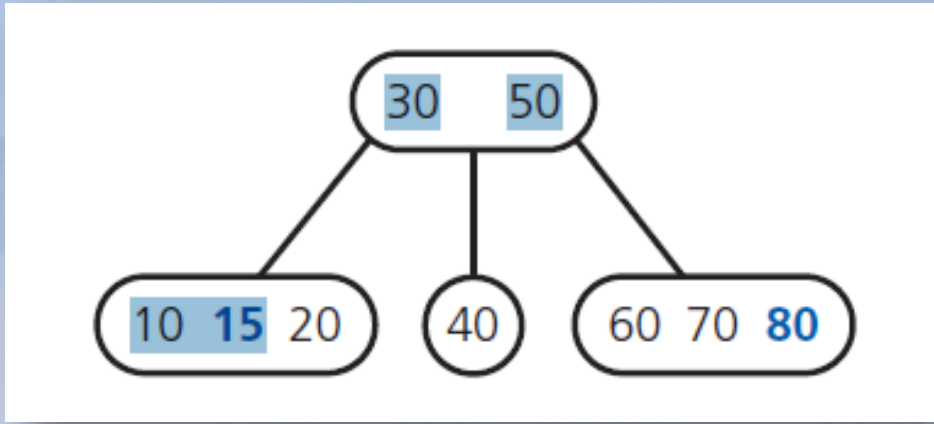


FIGURE 19-29 After adding 80 and 15 to the tree in Figure 19-28b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

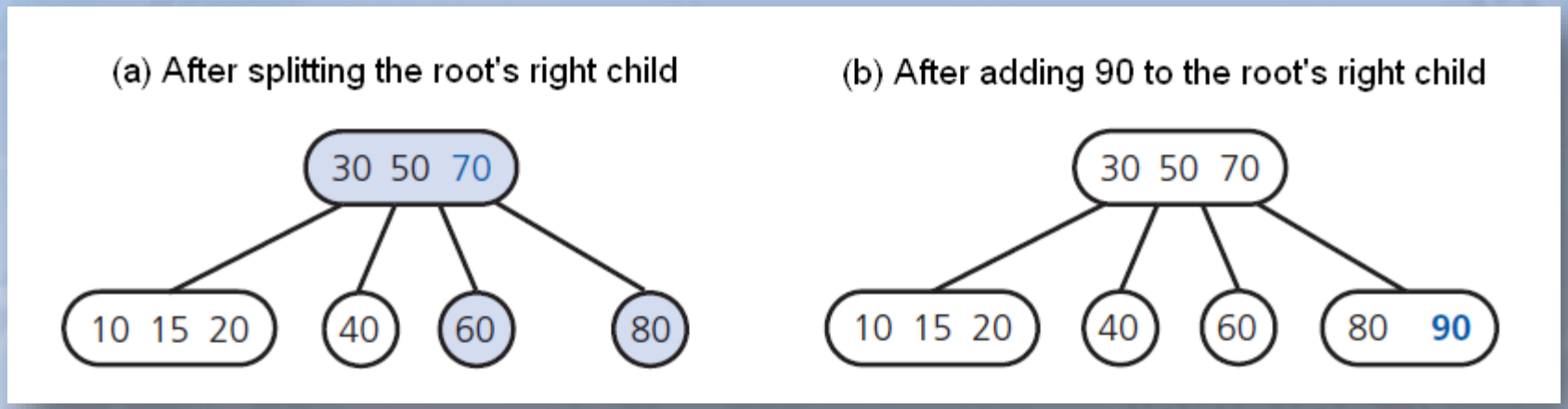


FIGURE 19-30 The steps for adding 90 to the tree in Figure 19-29

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

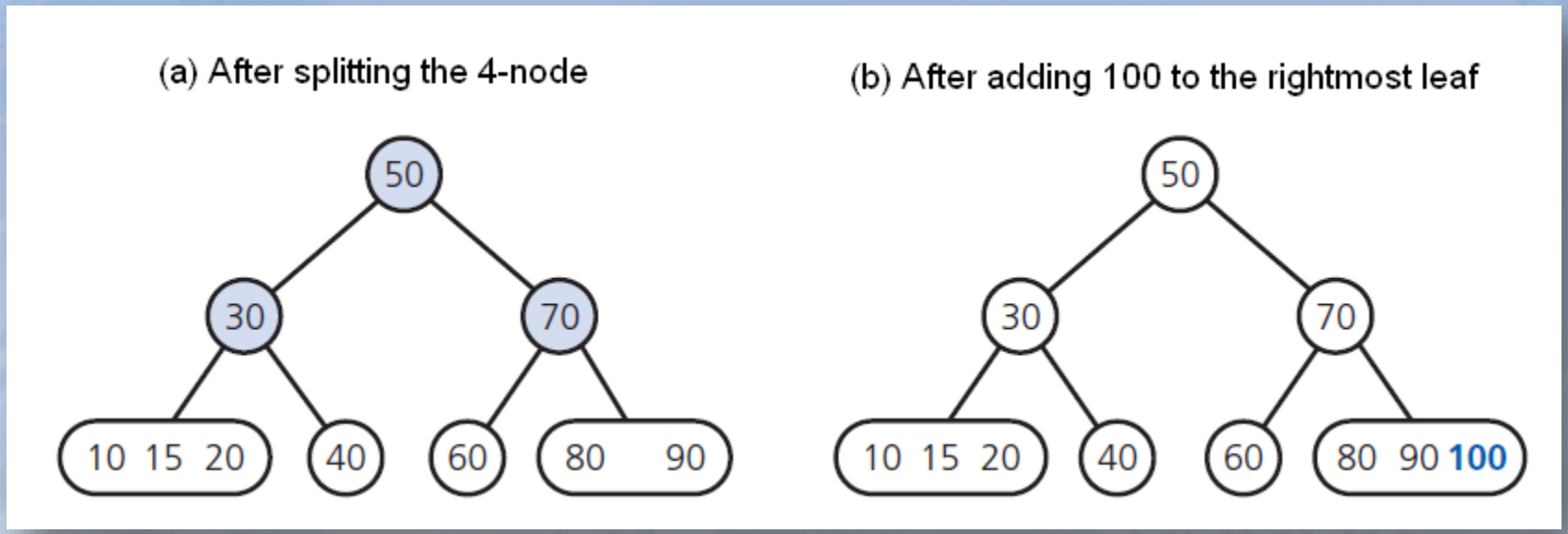


FIGURE 19-31 The steps for adding 100 to the tree in Figure 30b

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

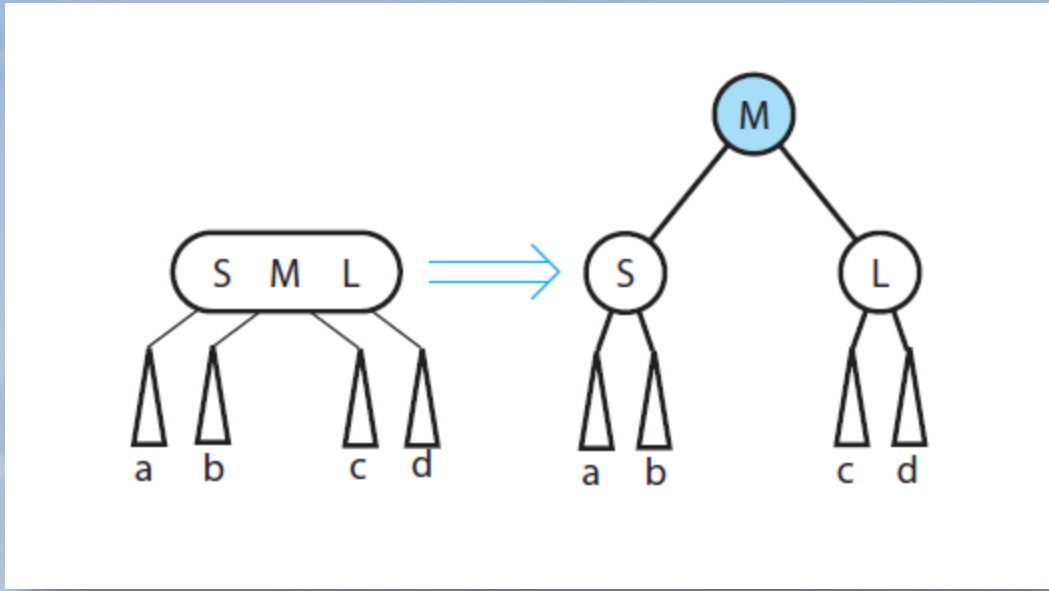


FIGURE 19-32 Splitting a 4-node root when adding data to a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

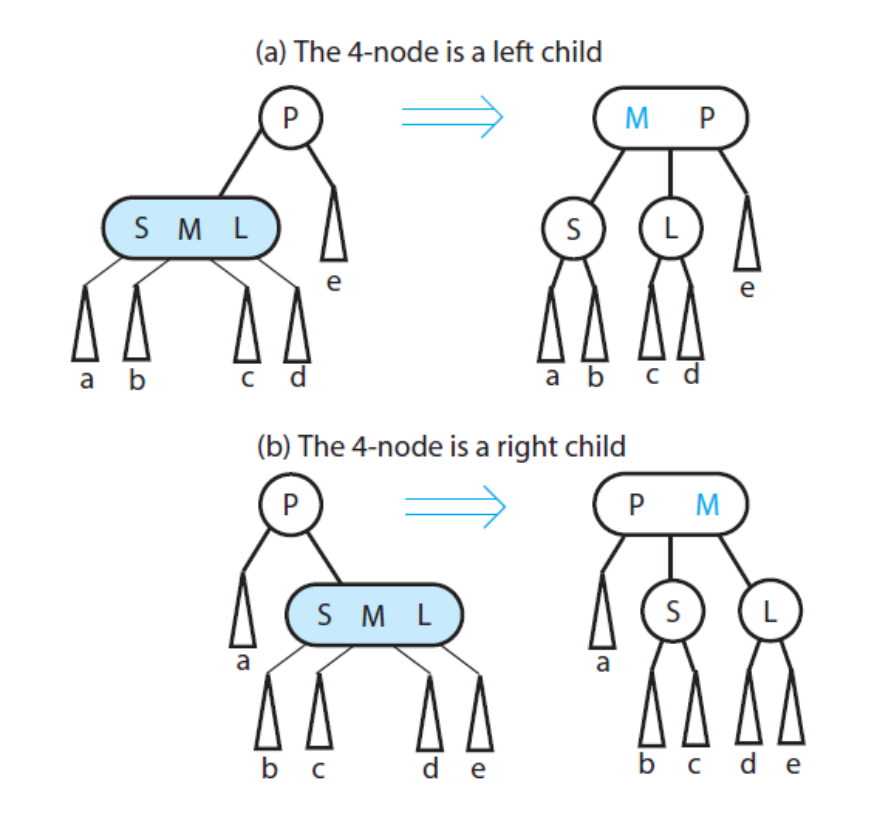


FIGURE 19-33 Splitting a 4-node whose parent is a 2-node when adding data to a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

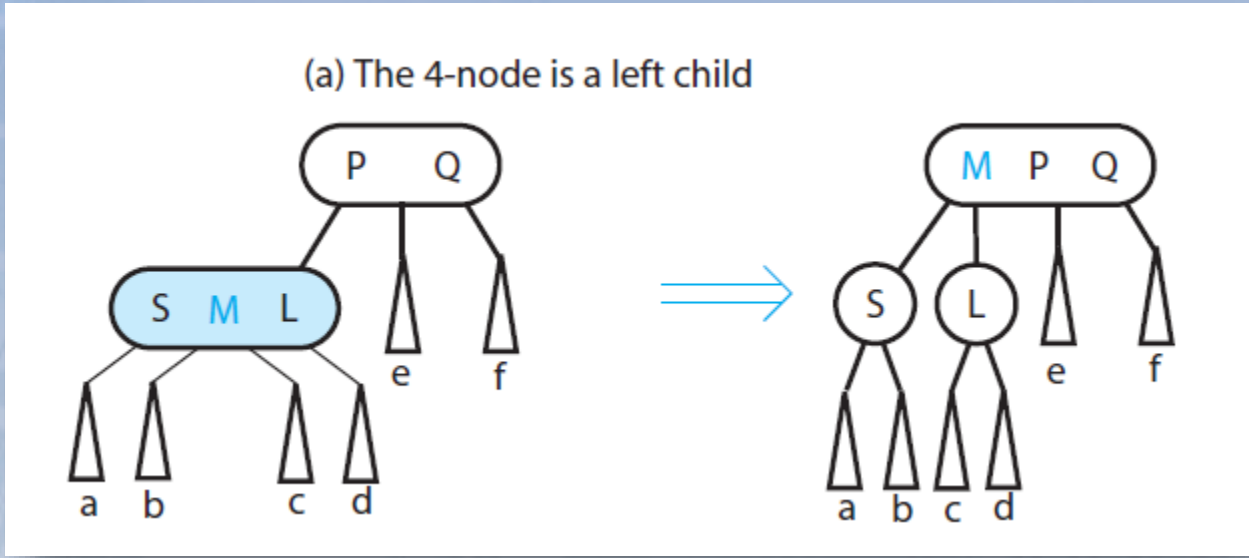


FIGURE 19-34 Splitting a 4-node whose parent is a 3-node when adding data to a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

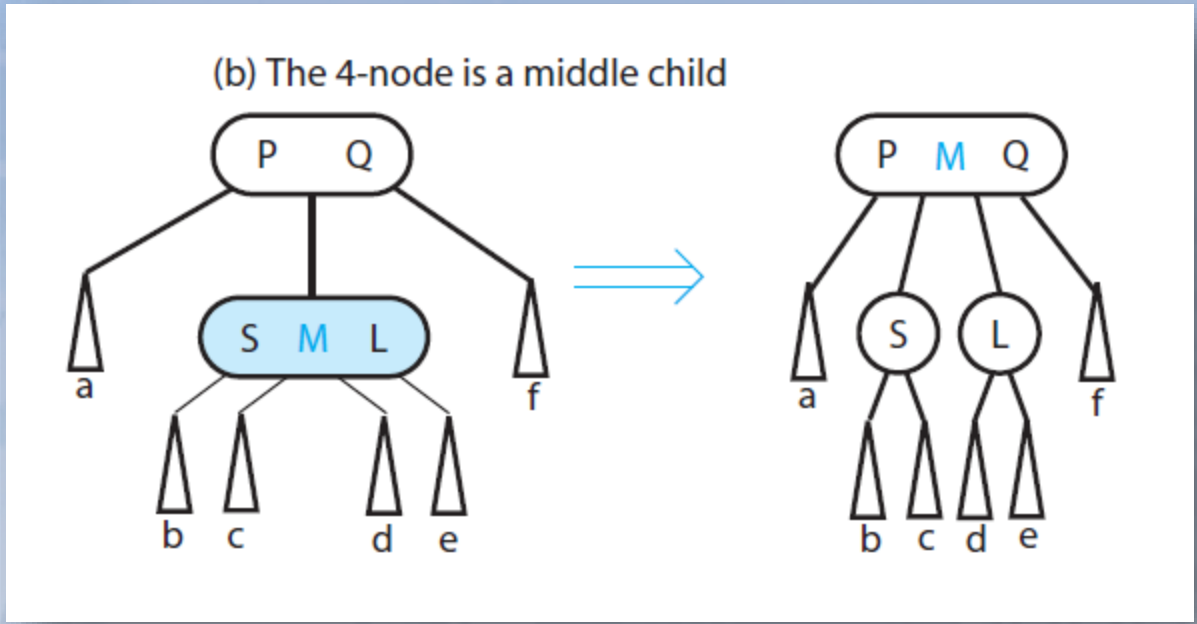


FIGURE 19-34 Splitting a 4-node whose parent is a 3-node when adding data to a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding Data to 2-3-4 Trees

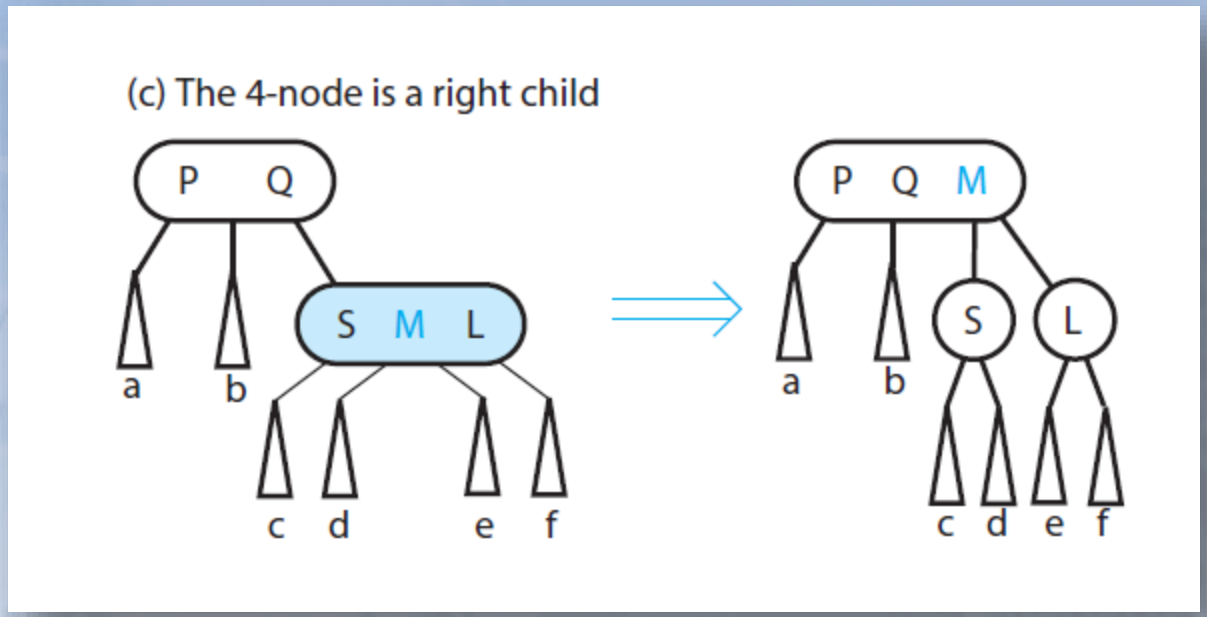


FIGURE 19-34 Splitting a 4-node whose parent is a 3-node when adding data to a 2-3-4 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Removing Data from a 2-3-4 Tree

- Has same beginning as removal algorithm for a 2-3 tree
- Transform each 2-node into a 3-node or a 4-node
- Insertion and removal algorithms for 2-3-4 tree require fewer steps than for 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

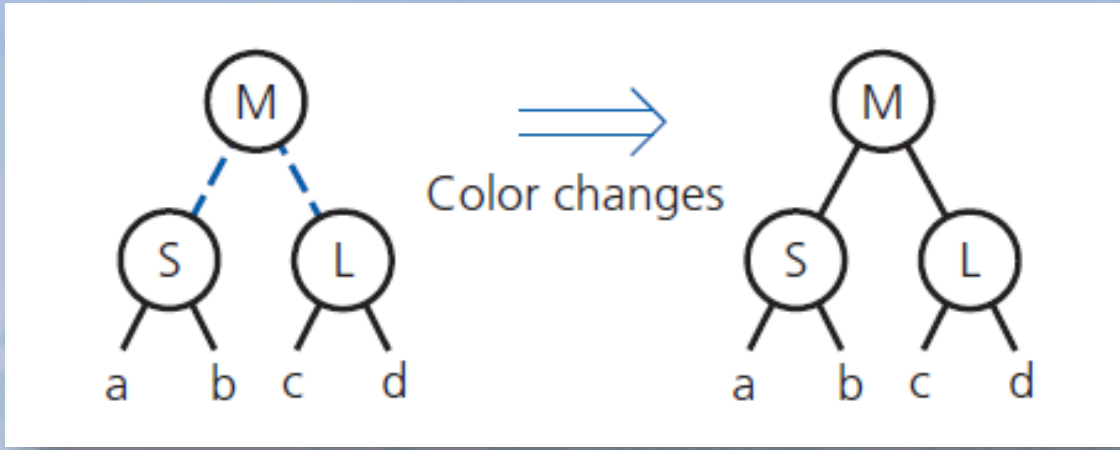


FIGURE 19-37 Splitting a red-black representation of a 4-node root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

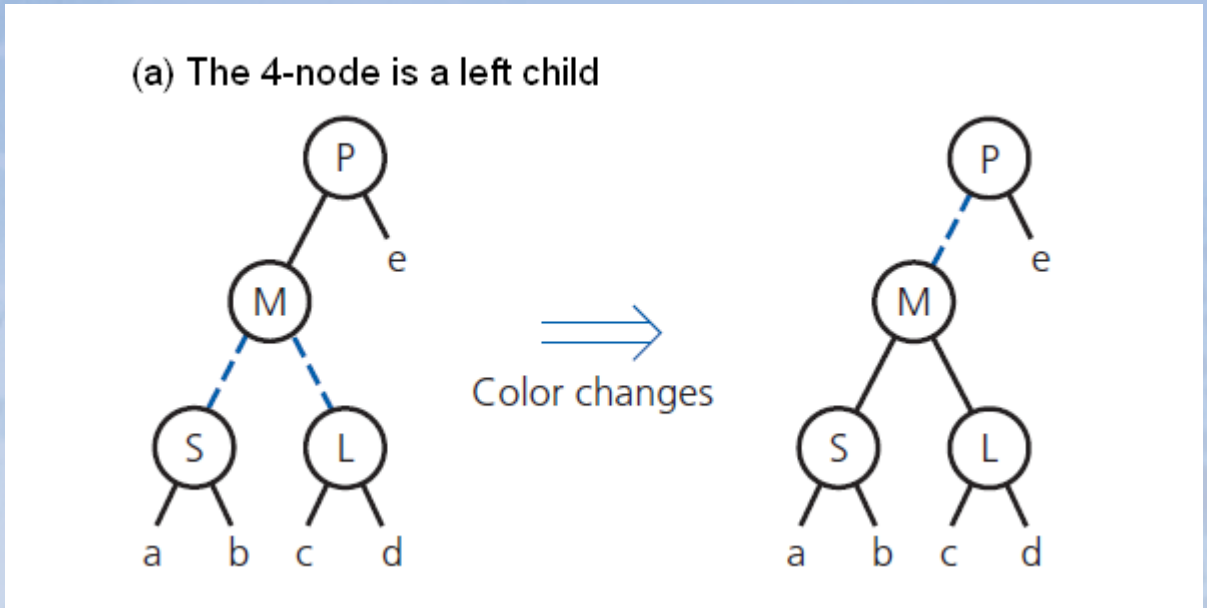


FIGURE 19-38 Splitting a red-black representation of a 4-node whose parent is a 2-node

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

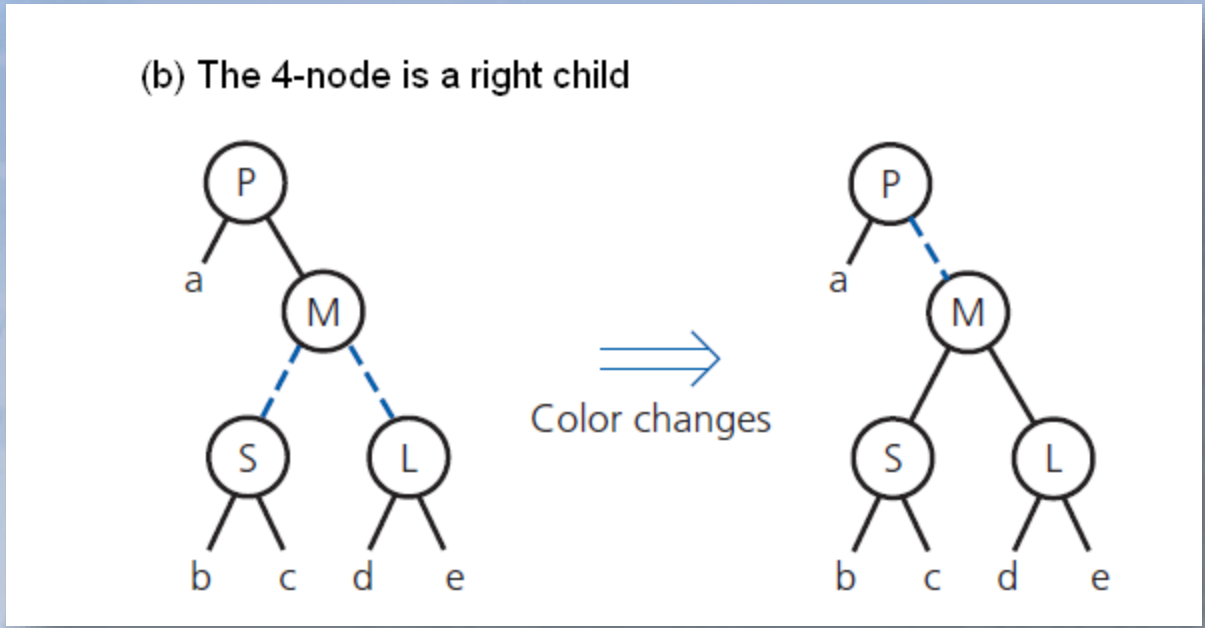
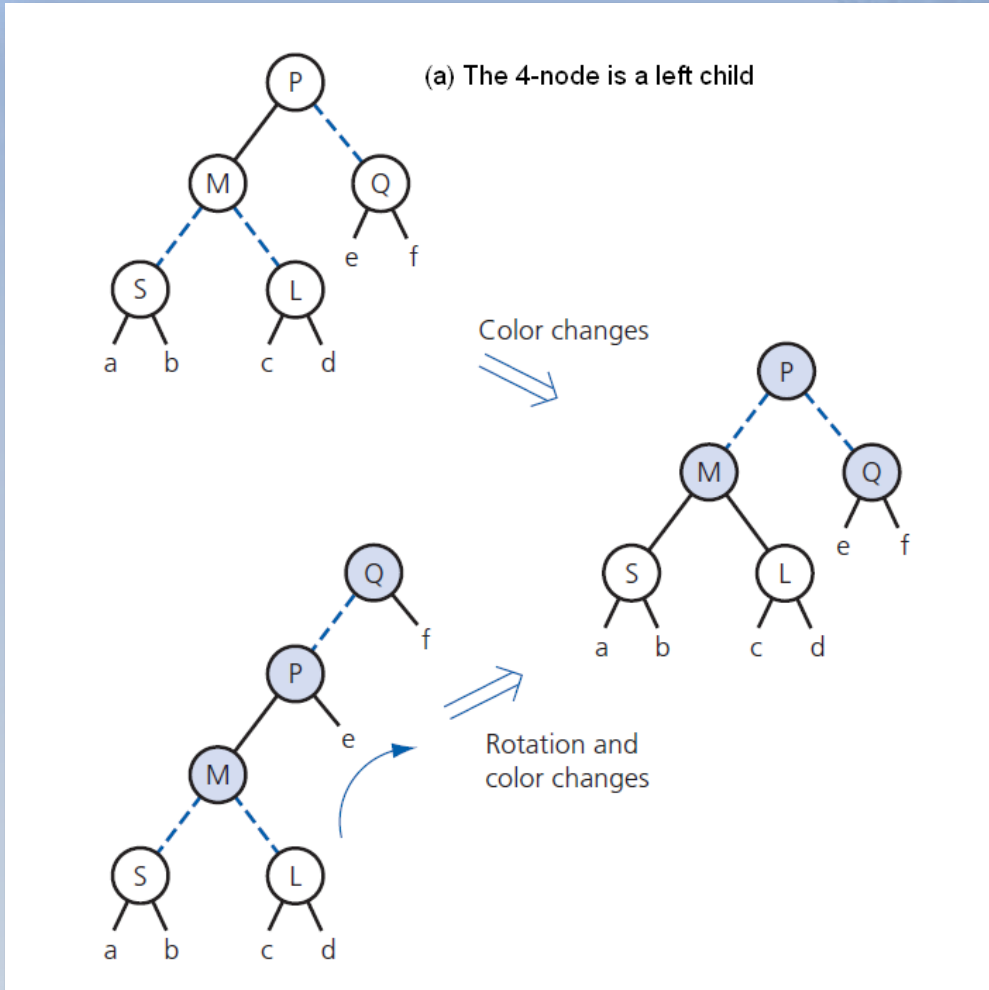


FIGURE 19-38 Splitting a red-black representation of a 4-node whose parent is a 2-node

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

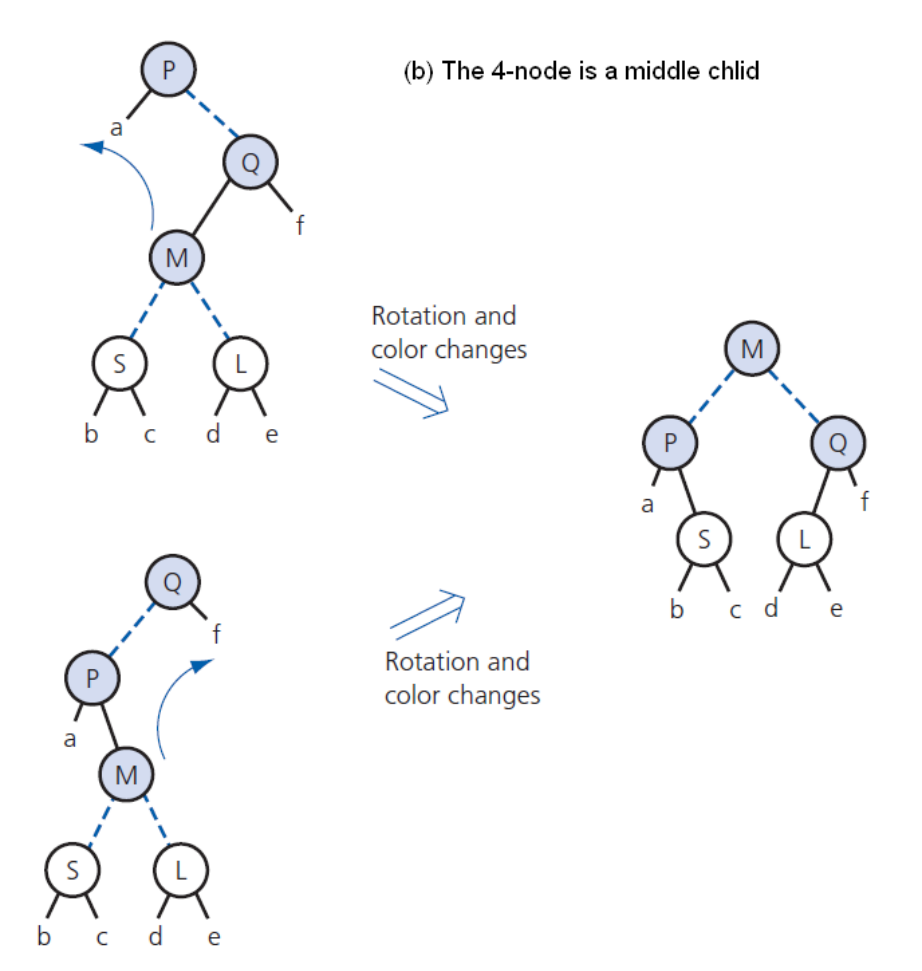
FIGURE 19-39 Splitting a red-black representation of a 4-node whose parent is a 3-node



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

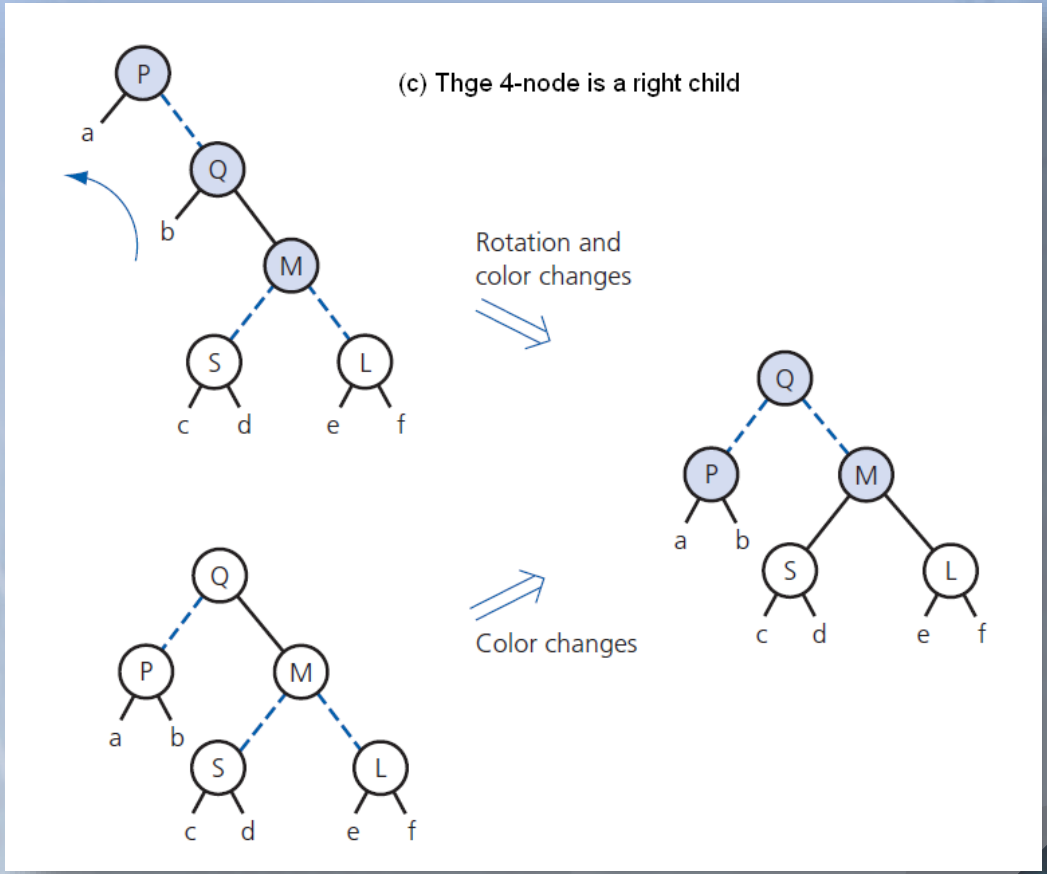
FIGURE 19-39 Splitting a red-black representation of a 4-node whose parent is a 3-node



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Adding to and Removing from a Red-Black Tree

FIGURE 19-39 Splitting a red-black representation of a 4-node whose parent is a 3-node



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

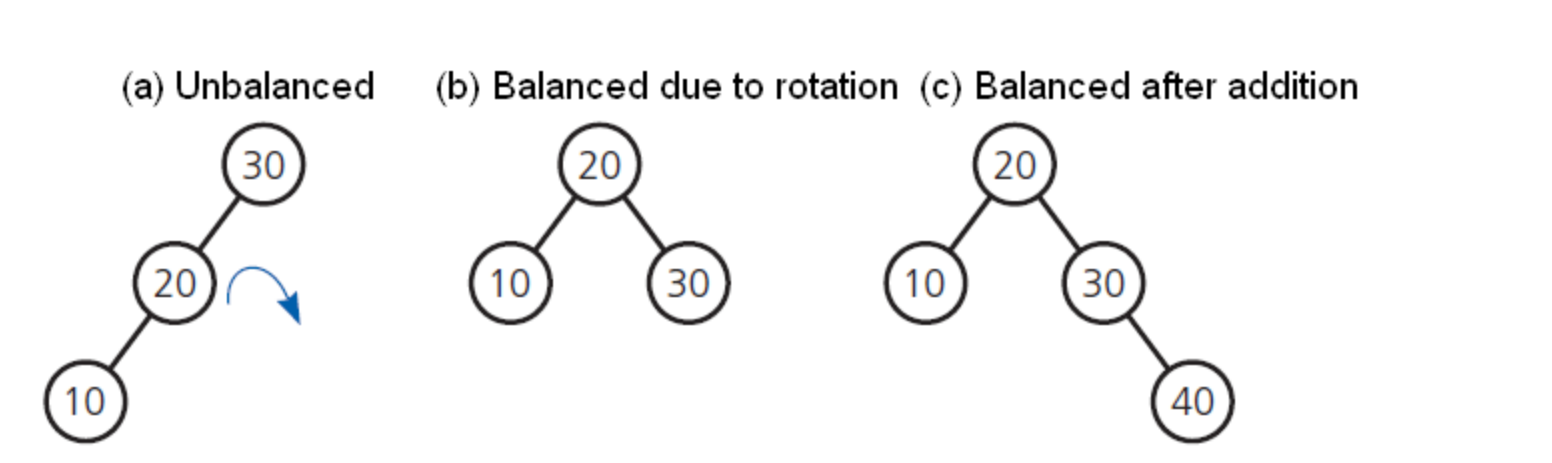


FIGURE 19-2 An unbalanced binary search tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

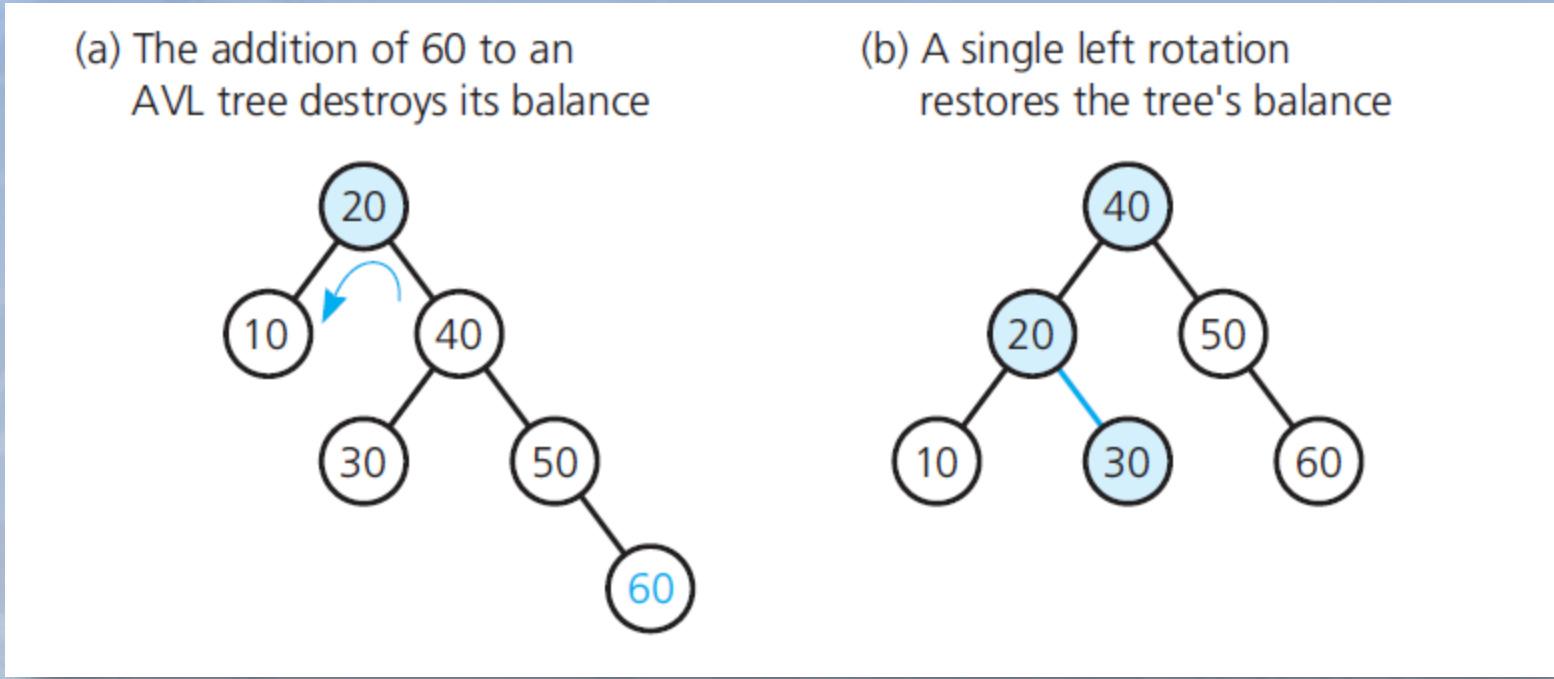


FIGURE 19-3 Correcting an imbalance in an AVL tree due to an addition by using a single rotation to the left

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

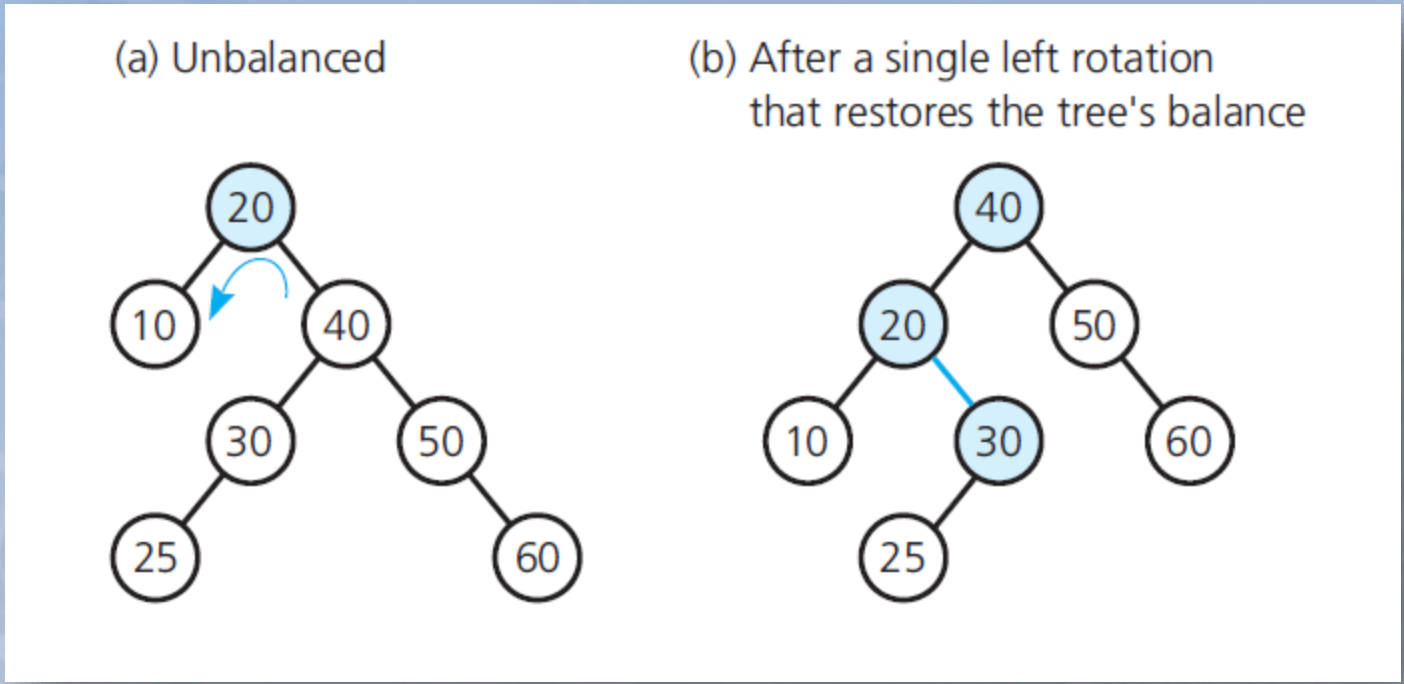


FIGURE 19-4 A single rotation to the left that does not affect the height of an AVL tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

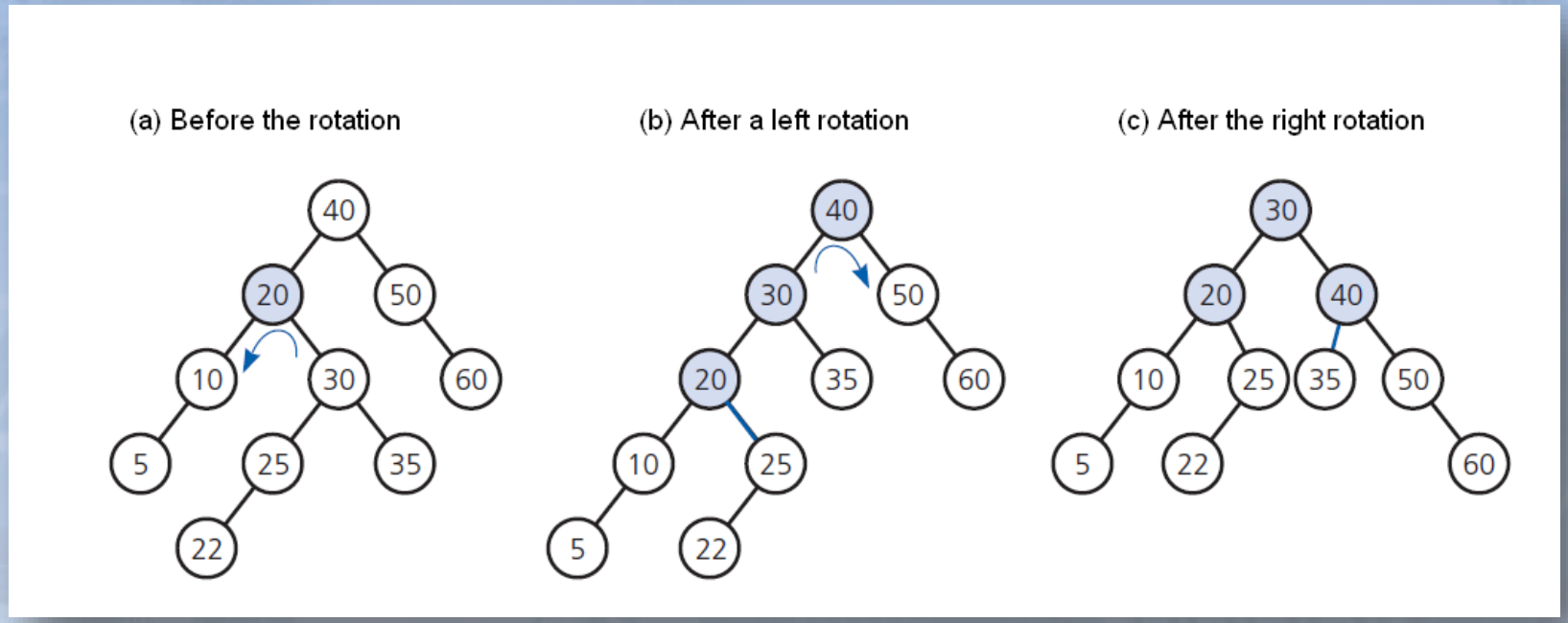


FIGURE 19-5 A double rotation that decreases the height of an AVL tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved